

Hands-On Exploratory Data Analytics

Prof. Dr. Nicolas Meseth

2. Juni 2026

Inhaltsverzeichnis

Vorwort	5
I. Vorbereitung	6
1. Eine Datenanalyseumgebung einrichten	7
1.1. Schritt 1: R installieren	7
1.2. Schritt 2: Python installieren	7
1.3. Schritt 3: Positron installieren	8
1.4. Schritt 4: Ein Projekt einrichten	8
1.5. Schritt 5: Quarto einrichten und testen	9
1.6. Schritt 6: Typst installieren (optional)	9
1.7. Schritt 7: Kleine Zusatzhelfer	10
1.8. Schritt 8: Reflexion	10
II. Tagesschau	12
2. Daten erkunden	13
2.1. Schritt 1: Projekt aufsetzen und Daten einlesen	13
2.2. Schritt 2: Erster Blick auf den Datensatz	14
2.3. Schritt 3: Strukturierte Variablen - Skalenniveaus und Datentypen	14
2.4. Schritt 4: Wertebereich und deskriptive Statistiken	15
2.5. Schritt 5: Unstrukturierte Variablen - Texte erkunden	15
2.6. Schritt 6: Reflexion	16
3. Daten transformieren	18
3.1. Schritt 1: Projekt aufsetzen und Daten einlesen	18
3.2. Schritt 2: Variablen auswählen und Zeilen filtern	19
3.3. Schritt 3: Neue Variablen erzeugen	20
3.4. Schritt 4: Daten gruppieren und zusammenfassen	22
3.5. Schritt 5: Datensätze verknüpfen mit Joins	23
4. Daten visualisieren	25
4.1. Schritt 1: Daten laden und vorbereiten	25
4.2. Schritt 2: Häufigkeiten und Verteilungen	25

4.3.	Schritt 3: Gruppenvergleiche visualisieren	26
4.4.	Schritt 4: Zeitliche Entwicklungen visualisieren	27
4.5.	Schritt 5: Zusammenhänge und mehrdimensionale Analysen	28
4.6.	Schritt 6: Fallstricke bei Visualisierungen	29
4.7.	Schritt 7: Reflexion	30
5.	Regelbasierte Textanalyse	32
5.1.	Schritt 1: Pakete laden und Daten einlesen	32
5.2.	Schritt 2: Texte durchsuchen und klassifizieren	32
5.3.	Schritt 3: Texte bereinigen und strukturieren	34
5.4.	Schritt 4: Strukturierte Textfelder parsen	35
5.5.	Schritt 5: Tokenisierung und Worthäufigkeiten	35
5.6.	Schritt 6: Wörterbuch-basierte Klassifikation und Reflexion	36
6.	LLM-gestützte Textanalyse	38
6.1.	Schritt 1: LLMs einrichten und testen	38
6.2.	Schritt 2: Analysecorpus aufbauen	39
6.3.	Schritt 3: Einfache NLP-Aufgaben mit LLMs	40
6.4.	Schritt 4: Ressort-Klassifikation und Evaluation	41
6.5.	Schritt 5: Sentiment-Analyse im Zeitverlauf	42
6.6.	Schritt 6: Wer spricht über wen?	43
6.7.	Schritt 7: Framing-Analyse mit LLMs	43
6.8.	Schritt 8: Reflexion	44
III.	Speisepläne	46
7.	Explorative Analyse von Speiseplänen in Deutschlands Mensen	47
7.1.	Hintergrund	47
7.2.	Forschungsfragen	47
7.3.	Informationen zum Datensatz	48
7.4.	Aufgabenstellung	49
7.4.1.	Teil 1: Zusammenführung und Bereinigung	49
7.4.2.	Teil 2: Einheitliche Klassifikation der Speisen	50
7.4.3.	Teil 3: Explorative Analysen	52
7.4.4.	Teil 4: Erstellung eines wissenschaftlichen Posters	52
7.5.	Bewertung	53
7.6.	Hinweise zur Fallstudie	53
7.6.1.	Bearbeitung	53
7.6.2.	Abgabe	54
7.6.3.	Postergalerie und Präsentation	55

IV. Umfrage	56
8. Variablen erkunden	57
8.1. Schritt 1: Projekt aufsetzen und Daten einlesen	57
8.2. Schritt 2: Erster Blick auf den Datensatz	58
8.3. Schritt 3: Einfache Einzelvariablen analysieren	58
8.4. Schritt 4: Mehrfachnennungen und Itembatterien	59
8.5. Schritt 5: Rangfolgen, Imagebatterien und Mediennutzung	59
8.6. Schritt 6: Reflexion	60
9. Gruppierte Betrachtung von Variablen	61
9.1. Schritt 1: Erste Schritte mit Gruppierungen	61
9.2. Schritt 2: Gruppierte Analysen und Visualisierungen	62
Literaturverzeichnis	64

Vorwort

Placeholder content for the second book.

Teil I.

Vorbereitung

1. Eine Datenanalyseumgebung einrichten

Positron, R, Python und Quarto

Bevor die eigentliche Datenanalyse und Schreibarbeit beginnen kann, braucht ihr eine funktionsfähige Arbeitsumgebung. Diese Übung führt euch Schritt für Schritt durch die Installation und Einrichtung aller benötigten Werkzeuge: von R und Python über die Entwicklungsumgebung Positron bis hin zu Quarto für reproduzierbare Berichte. Am Ende überprüft ihr mit einem kleinen Testszenario, dass alles reibungslos zusammenarbeitet, und denkt darüber nach, warum ihr diese Werkzeuge statt verbreiteter Alternativen wie Excel oder Word einsetzt.

1.1. Schritt 1: R installieren

R ist die primäre Sprache für diesen Kurs. Sie deckt den gesamten Analyseprozess ab: Daten einlesen, bereinigen, analysieren und visualisieren.

1. Öffnet die Website cran.r-project.org und ladet die aktuelle R-Version für euer Betriebssystem herunter. Führt das Installationsprogramm mit den Standardeinstellungen aus.

2. Überprüft die Installation: Öffnet ein Terminal (unter Windows z. B. PowerShell) und gebt `R.exe --version` ein. Welche Version wird angezeigt?

1.2. Schritt 2: Python installieren

Python verwenden wir als Ergänzung zu R, vor allem für den Zugriff auf KI-Modelle und Python-Bibliotheken, die keine direkte R-Entsprechung haben.

3. Ladet die neueste Version von Python von python.org herunter. Achtet bei der Installation unter Windows darauf, die Option *Add Python to PATH* zu aktivieren. Führt das

Installationsprogramm mit den Standardeinstellungen aus.

4. Öffnet ein Terminal und überprüft die Python-Installation mit `python --version` (unter macOS und Linux ggf. `python3 --version`). Stellt sicher, dass die angezeigte Version ≥ 3.12 ist.

1.3. Schritt 3: Positron installieren

Positron ist eine auf Data Science ausgerichtete Entwicklungsumgebung, die auf dem VS-Code-Unterbau aufbaut und R und Python gleichwertig unterstützt. Im Vergleich zu RStudio bietet Positron modernere Editor-Features und eine bessere Python-Integration.

5. Ladet Positron von positron.posit.co herunter und installiert es. Startet Positron nach der Installation und prüft in der Statusleiste unten im Fenster, ob eure R-Version automatisch erkannt wurde.

6. Macht euch mit der Benutzeroberfläche vertraut. Öffnet ein neues R-Skript (**File** → **New File** → **R File**) und führt `1 + 1` in der Konsole aus. Findet heraus, wo die Bereiche *Console*, *Variables* (Umgebungsanzeige) und *Explorer* (Dateimanager) zu finden sind. Wozu werden sie jeweils verwendet?

1.4. Schritt 4: Ein Projekt einrichten

Gut organisierte Projekte sind die Grundlage reproduzierbarer Analysen. Jedes Projekt bekommt seine eigene Ordnerstruktur und eine isolierte Paketumgebung über `renv`.

7. Erstellt einen neuen Ordner `setup-test` an einem sinnvollen Ort auf eurem Computer. Öffnet diesen Ordner in Positron über **File** → **Open Folder**. Überprüft in der R-Konsole mit `getwd()`, dass das Arbeitsverzeichnis auf den geöffneten Ordner zeigt.

8. Initialisiert eine isolierte Paketumgebung mit `renv`. Öffnet die R-Konsole im Projekt und führt `renv::init()` aus. Überprüft anschließend mit `renv::status()`, dass die Umgebung korrekt eingerichtet wurde.

9. Installiert die folgenden Grundpakete für alle Kurse: `tidyverse`, `janitor` und `skimr`. Verwendet das Metapaket `pacman`, das ihr zuerst installieren müsst, falls es noch nicht vorhanden ist. Überprüft nach der Installation, dass alle drei Pakete fehlerfrei geladen werden können.

1.5. Schritt 5: Quarto einrichten und testen

Quarto ist das System, mit dem ihr in diesem Kurs Analysen als reproduzierbare Berichte dokumentiert. In diesem Schritt erstellt ihr ein kleines Testdokument, das gleichzeitig sicherstellt, dass R, Pakete und Quarto korrekt zusammenarbeiten.

10. Überprüft, ob Quarto verfügbar ist. Öffnet das integrierte Terminal in Positron (**Terminal** → **New Terminal**) und gebt `quarto --version` ein. Quarto wird normalerweise zusammen mit Positron mitgeliefert.

11. Erstellt im Projektverzeichnis eine neue Datei `setup-test.qmd` (**File** → **New File** → **Quarto Document**). Tragt im YAML-Header den Titel "**Setup-Test**" ein und setzt `format: html`. Fügt dann einen R-Code-Block hinzu, der `tidyverse` lädt und ein Streudiagramm aus dem eingebauten Datensatz `mtcars` erzeugt: Fahrzeuggewicht (`wt`) auf der x-Achse und Kraftstoffverbrauch (`mpg`) auf der y-Achse.

12. Rendert das Dokument zu HTML: Klickt auf den *Render*-Button in der Werkzeugleiste oder führt im Terminal `quarto render setup-test.qmd` aus. Öffnet die erzeugte HTML-Datei und prüft, ob der Plot korrekt angezeigt wird.

13. Rendert das Testdokument ein zweites Mal als PDF: Ändert im YAML-Header `format: html` auf `format: pdf` und rendert erneut. Quarto installiert dabei ggf. automatisch TinyTeX (eine kompakte LaTeX-Distribution). Vergleicht das PDF mit der HTML-Version: Sehen beide inhaltlich identisch aus?

1.6. Schritt 6: Typst installieren (optional)

14. Ein weiteres spannendes Schreibwerkzeug, das textbasiert arbeitet, ist [Typst](#). Es bietet eine moderne, einfach zu erlernende Alternative zu LaTeX mit einer einfacheren Syntax und schnellerem Rendering. In diesem Buch werden wir hauptsächlich mit Quarto arbeiten,

weil es wunderbar mit der Arbeit mit Daten harmoniert. Wer aber neugierig ist, kann Typst als Ergänzung installieren und ausprobieren.

Typst ist lediglich ein Kommandozeilentool, das ein in der Sprache Typst geschriebenes Dokument in ein PDF rendern kann. Ihr könnt es auf verschiedene Weise installieren. [Hier geht es zum Download](#).

1.7. Schritt 7: Kleine Zusatzhelfer

Neben den Kernwerkzeugen gibt es einige Ergänzungen, die euch die tägliche Arbeit erheblich erleichtern.

15. Installiert Git von git-scm.com (Windows) oder über Homebrew (`brew install git`, macOS). Konfiguriert anschließend euren Namen und eure Hochschul-E-Mail und überprüft mit `git --version`, dass Git gefunden wird.

16. Aktualisiert den `renv`-Snapshot eures Projekts, um alle neu installierten Pakete in der Lock-Datei zu erfassen.

17. Informiert euch über KI-Assistenten für die Programmierung. Im Kurs werden wir gelegentlich *Claude Code* (Anthropic), *Codex* (OpenAI) oder *GitHub Copilot* (Microsoft) als VS-Code-Erweiterung verwenden. Recherchiert kurz, was die beiden Tools können, und notiert, wie sich KI-Assistenten eurer Meinung nach auf das Erlernen von Programmierung auswirken könnten, sowohl positiv als auch negativ.

1.8. Schritt 8: Reflexion

18. Vergleicht R mit Tabellenkalkulationsprogrammen wie Microsoft Excel. Welche Aufgaben lassen sich in Excel gut erledigen? Für welche Aufgaben ist R klar im Vorteil? Nennt mindestens zwei Stärken von R gegenüber Excel und überlegt, ob es auch Szenarien gibt, in denen Excel die bessere Wahl ist.

19. Vergleicht R (mit `ggplot2`) mit Visualisierungstools wie Tableau oder Power BI. Was können diese Tools, was R nur schwer kann? Und umgekehrt: Was spricht für den Einsatz von `ggplot2`?

20. Vergleicht Quarto mit Textverarbeitungsprogrammen wie Microsoft Word. Was sind die zentralen Unterschiede im Arbeitsablauf? Welche Vorteile bietet Quarto bei der Erstellung wissenschaftlicher oder technischer Berichte, welche Nachteile hat es?

21. Welche Bedeutung hat *Reproduzierbarkeit* für wissenschaftliche Analysen? Erläutert, wie die in diesem Experiment eingerichteten Werkzeuge (R, Quarto, renv, Git) gemeinsam dazu beitragen, dass Analysen reproduzierbar sind.

Teil II.

Tagesschau

2. Daten erkunden

Am Beispiel der Tagesschau-Daten

Nachrichtendatensätze enthalten eine interessante Mischung: Neben klar strukturierten Variablen wie Datum, Ressort oder Wortanzahl gibt es vollständig unstrukturierte Variablen wie Artikeltexte und Überschriften. Diese Übung führt euch durch eine systematische Erkundung dieses Datensatzes von der ersten Orientierung über die Analyse einzelner Variablen bis hin zu einfachen Methoden, wie man auf unstrukturierten Text eine Struktur projiziert.

2.1. Schritt 1: Projekt aufsetzen und Daten einlesen

1. Erstellt ein neues Projekt für diese Übung und öffnet es in unserer Entwicklungsumgebung Positron. Stellt sicher, dass ihr eine *virtuelle R-Umgebung* für das Projekt erstellt und aktiviert habt.

2. Installiert die folgenden Pakete in eurer virtuellen Umgebung: `tidyverse`, `janitor` und `skimr`. Verwendet das Metapaket `pacman`, damit ihr die Pakete mit einem einzigen Befehl installieren und laden könnt.

3. Ladet den Datensatz `tagesschau.zip` auf euren Computer herunter und speichert ihn in einem Unterordner `data/` in eurem Projektverzeichnis.

4. Erzeugt einen neuen Ordner `scripts/` in eurem Projektverzeichnis und erstellt eine neue R-Skriptdatei `explore_news.R` in diesem Ordner.

5. Der Datensatz liegt als komprimierte ZIP-Datei vor. Schaut euch die Datei genauer an und prüft, wie das Format aussieht (z. B. Trennzeichen, Zeichenkodierung). Lest die Daten dann mit einer geeigneten Funktion aus `readr` ein und speichert den Code in eurer

Skriptdatei.

2.2. Schritt 2: Erster Blick auf den Datensatz

6. Recherchiert den Hintergrund der Daten: Woher stammen sie, wie wurden sie erhoben, und welchen Zeitraum decken sie ab? Was bedeuten die einzelnen Variablen? Haltet eure Erkenntnisse stichpunktartig fest.

7. Verschafft euch mit R einen ersten Überblick. Nutzt dafür mindestens zwei verschiedene Funktionen (z. B. `glimpse()`, `skim()`, `head()`). Beantwortet dabei: Wie viele Zeilen und Spalten hat der Datensatz? Welche Datentypen wurden beim Einlesen automatisch zugewiesen?

8. Jede Beobachtung in einem Datensatz sollte eindeutig identifizierbar sein. Welche Variable könnte hier als eindeutiger Identifikator dienen? Überprüft eure Vermutung mit R, z. B. mithilfe von `n_distinct()` oder `get_dupes()` aus dem `janitor`-Paket. Was macht ihr, wenn keine bestehende Variable geeignet ist?

2.3. Schritt 3: Strukturierte Variablen - Skalenniveaus und Datentypen

In diesem Abschnitt konzentriert ihr euch auf die Variablen, die klare Messeigenschaften besitzen.

9. Betrachtet die folgenden Variablen: `ressort`, `tag`, `language`, `word_count`, `date_time`. Ordnet jeder Variable ein Skalenniveau zu (nominal, ordinal, intervall oder verhältnisskaliert). Begründet eure Entscheidung in einem kurzen Satz pro Variable.

10. Prüft, ob die von R automatisch zugewiesenen Datentypen zu den Skalenniveaus passen. Bei welchen Variablen gibt es eine Abweichung? Welche Variablen sollten als Faktor (`factor`) gespeichert werden, und warum?

11. Wandelt `ressort`, `tag` und `language` in Faktoren um. Nutzt dafür `mutate()` in Kombination mit `as_factor()` oder `factor()`. Schaut euch anschließend mit `levels()` an, welche Ausprägungen jeweils vorhanden sind. Wie viele Ressorts gibt es insgesamt?

12. Die Variable `tag` enthält viele fehlende Werte (`NA`). Ermittelt den Anteil der fehlenden Werte in Prozent. Was könnte ein fehlender `tag`-Wert inhaltlich bedeuten? Ist er wirklich “unbekannt”, oder könnte er auch “kein Tag vergeben” bedeuten? Wie würdet ihr damit umgehen?

2.4. Schritt 4: Wertebereich und deskriptive Statistiken

13. Welchen Zeitraum deckt der Datensatz ab? Ermittelt das früheste und das späteste Datum in `date_time`. Extrahiert außerdem das Erscheinungsjahr mit `lubridate::year()` und erstellt eine neue Variable `year`. Wie hat sich die Anzahl der Artikel pro Jahr entwickelt?

14. Berechnet für `word_count` passende deskriptive Kenngrößen. Welche Kenngrößen sind für eine verhältnisskalierte Variable sinnvoll? Berechnet Minimum, Maximum, Median, Mittelwert und Standardabweichung. Vergleicht Median und Mittelwert - was sagt euch der Unterschied beider Werte über die Verteilung?

15. Erstellt für `word_count` ein einfaches Histogramm mit `ggplot2`. Wählt eine sinnvolle Anzahl an Balken (`bins`). Beschreibt die Verteilung: Ist sie symmetrisch, linksschief oder rechtsschief? Gibt es Ausreißer?

16. Für nominalskalierte Variablen ist der Modus die geeignete Maßzahl. Ermittelt die häufigste Ausprägung (*Modus*) für `ressort` und für `tag`. Stellt die Häufigkeitsverteilung der Ressorts anschließend als Balkendiagramm dar. Sortiert die Balken nach Häufigkeit.

17. Berechnet die mittlere Wortanzahl getrennt nach `ressort`. Welches Ressort produziert im Schnitt die längsten Artikel? Stellt das Ergebnis als horizontales Balkendiagramm dar.

2.5. Schritt 5: Unstrukturierte Variablen - Texte erkunden

18. Betrachtet die Variablen `title`, `shorttext` und `text`. Was unterscheidet sie von den bisher analysierten Variablen? Warum lassen sich auf diese Variablen keine klassischen deskriptiven Statistiken wie Mittelwert oder Median anwenden?

19. Eine einfache Form von Struktur ist die Länge eines Textes. Berechnet mit `str_length()` die Zeichenlänge der Variablen `title` und speichert sie als neue Variable `title_length`. Berechnet dann Minimum, Median und Maximum. Was ist der kürzeste und was der längste Titel im Datensatz?

20. Nutzt `str_count()` mit einem Leerzeichen als Muster (" "), um die Anzahl der Wörter in `title` zu schätzen. Speichert das Ergebnis als `title_word_count`. Wie viele Wörter hat ein typischer Tagesschau-Titel im Median?

21. Texte lassen sich auch durch das Suchen nach bestimmten Mustern (Schlüsselwörter) strukturieren. Erstellt mit `str_detect()` eine logische Variable `mentions_ukraine`, die angibt, ob der Titel das Wort "Ukraine" enthält. Wie viele Artikel erwähnen die Ukraine im Titel? In welchem Jahr gab es die meisten solchen Artikel? Wie ist die zeitliche Entwicklung?

22. Untersucht auf dieselbe Weise ein zweites Thema eurer Wahl (z. B. "Klimawandel", "Corona", "USA", "Bundestagswahl", "Iran"). Zählt die Treffer pro Jahr und stellt die zeitliche Entwicklung als Liniendiagramm dar. Wie könnt ihr beide Themen in einem Diagramm vergleichen? Welche Erkenntnisse könnt ihr daraus ziehen?

2.6. Schritt 6: Reflexion

23. Fasst den Unterschied zwischen strukturierten und unstrukturierten Variablen in eigenen Worten zusammen. Welche Analyseschritte waren für beide Variablentypen möglich, und welche nur für strukturierte?

24. Welche Einschränkungen hat die einfache Textanalyse mit `str_detect()` im Vergleich zu einer vollwertigen inhaltsanalytischen Auswertung? Nennt mindestens zwei Probleme, die bei der Schlüsselwortsuche auftreten können.

25. Betrachtet die Variable `ressort`. Sie wurde beim Einlesen als `character` gespeichert, aber ihr habt sie in einen Faktor umgewandelt. Welchen praktischen Unterschied macht das bei der Analyse und Visualisierung in R?

26. Der Datensatz enthält Artikel aus fast zwei Jahrzehnten. Welche Probleme könnten bei einem Vergleich von Artikeln aus dem Jahr 2006 mit solchen aus dem Jahr 2024 entstehen? Denkt dabei auch an mögliche Veränderungen im Datensatz oder der Datenquelle selbst.

3. Daten transformieren

Am Beispiel der Tagesschau-Daten

Rohdaten sind selten analysebereit. Bevor ihr eine Visualisierung erstellt, müsst ihr auswählen, was ihr braucht, irrelevantes herausfiltern, neue Variablen ableiten, Gruppen zusammenfassen und verschiedene Tabellen verknüpfen. Genau das ist Datentransformation, und **dplyr** liefert dafür ein konsistentes, lesbares Vokabular aus wenigen, aber mächtigen Verben. In dieser Übung lernt ihr diese Verben am Tagesschau-Datensatz kennen und baut dabei Schritt für Schritt ein vollständiges Transformations-Repertoire auf.

3.1. Schritt 1: Projekt aufsetzen und Daten einlesen

Die Aufgaben 1 bis 4 sind für euch nur relevant, wenn ihr noch kein Projekt angelegt habt. Wenn ihr bereits ein Projekt für eine vorherige Übung erstellt habt, könnt ihr die Aufgaben 1 bis 3 sowie 5 auslassen und nur das neue R-Skript in Aufgabe 4 anlegen.

1. Erstellt ein neues Projekt für diese Übung und öffnet es in unserer Entwicklungsumgebung Positron. Stellt sicher, dass ihr eine *virtuelle R-Umgebung* für das Projekt erstellt und aktiviert habt.

2. Installiert die folgenden Pakete in eurer virtuellen Umgebung: **tidyverse**, **janitor**, **skimr**, **ggribes** und **tidylog**. Verwendet das Metapaket **pacman**, damit ihr die Pakete mit einem einzigen Befehl installieren und laden könnt.

3. Ladet den Datensatz **tagesschau.zip** auf euren Computer herunter und speichert ihn in einem Unterordner **data/** in eurem Projektverzeichnis.

4. Erzeugt einen neuen Ordner **scripts/** in eurem Projektverzeichnis und erstellt eine neue R-Skriptdatei **transform_news.R** in diesem Ordner.

5. Der Datensatz liegt als komprimierte ZIP-Datei vor. Schaut euch die Datei genauer an und prüft, wie das Format aussieht, zum Beispiel Trennzeichen und Zeichenkodierung. Lest die Daten dann mit einer geeigneten Funktion aus `readr` ein und speichert den Code in eurer Skriptdatei.

3.2. Schritt 2: Variablen auswählen und Zeilen filtern

`select()`, `filter()` und `arrange()` sind die drei grundlegenden Verben, um einen Datensatz einzugrenzen: Was brauche ich? Welche Zeilen sind relevant? In welcher Reihenfolge sollen sie erscheinen? Zusammen mit `tidylog` habt ihr außerdem ein Werkzeug, das euch bei jeder Transformation transparent zeigt, was tatsächlich passiert.

6. Das Paket `tidylog` überwacht dplyr-Operationen still im Hintergrund und gibt nach jeder Transformation eine Meldung aus, wie viele Zeilen entfernt wurden, wie viele neue Variablen entstanden sind und wie die Gruppen aussehen. Es muss *nach* dem Laden von `tidyverse` geladen werden, damit es die dplyr-Funktionen korrekt überschreibt.

Ladet `tidylog` mit `library(tidylog)` und führt dann probeweise folgende Befehle aus:

```
filter(news, ressort == "inland")
filter(news, word_count > 2000)
select(news, title, ressort, word_count)
```

Was steht in den Meldungen von `tidylog`? Welche Information bekommt ihr, die dplyr allein euch nicht geben würde? In welchen Situationen ist `tidylog` besonders hilfreich?

7. Der Datensatz enthält 21 Variablen, aber für viele Analysen braucht ihr davon nur einen kleinen Teil. Erstellt einen reduzierten Datensatz `news_slim`, der ausschließlich die Variablen `title`, `date_time`, `ressort`, `tag`, `author` und `word_count` enthält. Verwendet dafür `select()`. Beobachtet, was `tidylog` dabei meldet. Wie viele Zeilen hat `news_slim`?

8. `select()` kann Spalten auch umbenennen, indem ihr `neuer_name = alter_name` schreibt. Das ist praktisch, wenn ihr gleichzeitig auswählt und umbenennt. Wenn ihr jedoch *alle* Spalten behalten und nur eine oder wenige umbenennen wollt, ist `rename()` die bessere Wahl.

Benennt in `news_slim` die Spalte `word_count` mit `rename()` in `n_words` um und nennt das Ergebnis `news_renamed`. Wie würde dieselbe Umbenennung mit `select()` aussehen? Was müsstet ihr dabei zusätzlich angeben? Überschreibt `news_slim` *nicht*, da spätere Aufgaben den ursprünglichen Namen `word_count` erwarten.

9. Filtert `news_slim` auf alle Artikel aus dem Ressort "inland" und speichert das Ergebnis als `news_inland`. Wie viele Inland-Artikel enthält der Datensatz? Was sagt `tidylog` dazu?

10. Häufig möchtet ihr nach mehreren erlaubten Werten gleichzeitig filtern. Der Operator `%in%` prüft, ob ein Wert in einem Vektor enthalten ist, und vermeidet mehrere `|`-Bedingungen.

Definiert zunächst einen Vektor `hauptressorts` mit den sechs häufigsten Ressorts: "inland", "ausland", "wirtschaft", "wissen", "investigativ" und "faktenfinder". Filtert `news_slim` mit `%in%` auf diese Ressorts und speichert das Ergebnis als `news_haupt`. Wie viele Zeilen enthält es? Schreibt daneben, wie der äquivalente Filter mit `|` aussehen würde, und entscheidet, welche Version ihr lesbarer findet.

11. Wie viele Artikel aus dem Ressort "ausland" wurden seit dem 1. Januar 2020 veröffentlicht *und* haben mehr als 800 Wörter? Von welchem Datum stammt der älteste dieser Artikel?

12. Die Variable `author` enthält für viele Artikel keine Angabe. Ermittelt zunächst, welchen Anteil in Prozent die Artikel ohne Autorengabe am Gesamtdatensatz ausmachen. Erstellt dann eine gefilterte Version `news_with_author`, die nur Zeilen *mit* Autorengabe enthält.

Was könnte es inhaltlich bedeuten, wenn kein Autor angegeben ist? Beobachtet, wie `tidylog` den NA-Filter beschreibt.

13. Welche zehn Artikel haben die meisten Wörter? Verwendet `arrange()` mit `desc()` und `head()`, oder alternativ `slice_max()`, um die zehn längsten Artikel zu identifizieren. Gebt `title`, `ressort` und `word_count` aus. In welchen Ressorts erscheinen die besonders langen Artikel?

3.3. Schritt 3: Neue Variablen erzeugen

Mit `mutate()` könnt ihr bestehende Datensätze um neue, berechnete Variablen erweitern, ohne dabei andere Spalten oder die Zeilenanzahl zu verändern. In diesem Teil lernt ihr außerdem, wie ihr Datentypen anpasst, Datumsvariablen zerlegt, kategoriale Einteilungen vornehmt und mit `across()` dieselbe Transformation auf viele Spalten gleichzeitig anwendet.

14. Die Variable `date_time` enthält sowohl Datum als auch Uhrzeit. Für viele Analysen brauchen wir sie in Einzelteilen. Fügt dem Datensatz `news_slim` mit `mutate()` drei neue

Variablen hinzu:

- `year`, das Erscheinungsjahr
- `month`, den Erscheinungsmonat als Zahl von 1 bis 12
- `weekday`, den Wochentag als geordneter Faktor, Funktion: `wday(..., label = TRUE)`

Speichert den erweiterten Datensatz wieder als `news_slim`. Welche Wochentage sind im Datensatz vertreten, und wie heißen sie in R?

15. Die Variable `ressort` wurde beim Einlesen als `character` gespeichert, ist inhaltlich aber eine nominale kategoriale Variable. Wandelt sie mit `mutate()` und `as_factor()` in einen Faktor um. Überprüft mit `levels()`, welche Ausprägungen vorhanden sind, und mit `nlevels()`, wie viele es insgesamt gibt.

Macht dasselbe mit der Variable `tag`. Wie viele verschiedene Tags enthält der Datensatz? Fällt euch dabei etwas Unerwartetes auf?

16. Auch aus Textvariablen können wir nützliche Kenngrößen ableiten. Erstellt mit `mutate()` und Funktionen aus `stringr` zwei neue Variablen:

- `title_length`, die Zeichenanzahl des Titels
- `title_word_count`, die geschätzte Wortanzahl des Titels, zählt dafür die Leerzeichen und addiert 1

Berechnet für `title_length` Minimum, Median und Maximum. Was ist der kürzeste und was der längste Titel im Datensatz? Druckt die Artikel mit dem kürzesten und dem längsten Titel auf der Konsole aus.

17. Die Funktion `transmute()` verhält sich wie `mutate()`, behält aber im Ergebnis nur die neu erzeugten Spalten. Erstellt mit `transmute()` einen neuen kompakten Datensatz `news_timeline`, der ausschließlich folgende Variablen enthält: `year`, `month`, `weekday`, `ressort`, `word_count` sowie `title_length` aus Aufgabe 16. Speichert diesen Datensatz für spätere Analysen.

In welchen Situationen ist `transmute()` praktischer als `mutate()`? Nennt ein konkretes Beispiel.

18. Metrische Variablen lassen sich sinnvoll in inhaltliche Kategorien einteilen. Erstellt mit `mutate()` und `case_when()` eine neue Variable `article_type`:

- "Kurzmeldung", weniger als 200 Wörter

- "Standardartikel", 200 bis unter 700 Wörter
- "Langer Artikel", 700 Wörter und mehr

Wandelt `article_type` anschließend in einen Faktor mit inhaltlich sinnvoller Reihenfolge um. Berechnet, wie viele Artikel auf jede Kategorie entfallen, und ermittelt den Anteil in Prozent.

19. `across()` erlaubt es, dieselbe Transformation auf mehrere Spalten gleichzeitig anzuwenden, ohne den Code zu wiederholen. Es funktioniert sowohl in `mutate()` als auch in `summarize()`.

Löst die folgenden beiden Aufgaben mit `across()`:

- Berechnet in einem einzigen `summarize()`-Aufruf Mittelwert und Median sowohl für `word_count` als auch für `title_length`. Nutzt dafür `across(c(word_count, title_length), list(mean = ~mean(.x, na.rm = TRUE), median = ~median(.x, na.rm = TRUE)))`.
- Wendet in einem einzigen `mutate()`-Aufruf `str_trim()` auf alle Zeichenketten-Spalten in `news_slim` an. Nutzt dafür `across(where(is.character), str_trim)`.

Wie viele Zeilen Code hätte jede Lösung ohne `across()` gebraucht?

3.4. Schritt 4: Daten gruppieren und zusammenfassen

`group_by()` und `summarize()` sind das mächtigste Werkzeugpaar in `dplyr`. Mit ihnen berechnet ihr beliebige Kenngrößen für beliebige Gruppen. Wichtig ist dabei, den Unterschied zwischen `summarize()` und `mutate()` nach `group_by()` zu verstehen, sowie die Konsequenzen, die es hat, `ungroup()` zu vergessen.

20. Berechnet die Anzahl der Artikel pro Ressort und sortiert das Ergebnis absteigend nach Häufigkeit. Welches ist das häufigste Ressort? Welche Ressorts enthalten weniger als 100 Artikel? Notiert euch die sechs häufigsten Ressorts als `hauptressorts`-Vektor für spätere Analysen.

21. Berechnet für die sechs Hauptressorts folgende Kenngrößen in einem einzigen `summarize()`-Aufruf:

- Anzahl der Artikel, `n`
- Median der Wortanzahl, `median_word_count`
- Mittlere Wortanzahl, gerundet auf ganze Zahlen, `mean_word_count`

- Anteil der Artikel mit namentlichem Autor in Prozent. Ein Autor gilt als namentlich, wenn `author` weder `NA` noch `"tagesschau.de"` ist.

Sortiert das Ergebnis absteigend nach der mittleren Wortanzahl. Welches Ressort produziert im Schnitt die längsten Artikel?

22. Wie hat sich die Artikelanzahl über die Jahre entwickelt? Berechnet für jedes Jahr die Gesamtanzahl aller Artikel sowie separat die Anzahl für die drei Ressorts `"inland"`, `"ausland"` und `"wirtschaft"`.

Fällt euch etwas an den Jahren 2006 und 2026 auf? Warum könnten diese Jahrgänge unvollständig sein, und wie solltet ihr das bei Interpretationen berücksichtigen?

23. `group_by()` funktioniert nicht nur mit `summarize()`, sondern auch mit `mutate()`. Während `summarize()` den Datensatz auf eine Zeile pro Gruppe verdichtet, behält `mutate()` alle Originalzeilen und fügt nur eine neue Spalte hinzu.

Fügt dem auf die Hauptressorts gefilterten Datensatz mit `group_by()` und `mutate()` zwei neue Variablen hinzu:

- `ressort_median_wc`, der Median der Wortanzahl *des jeweiligen Ressorts*
- `above_median`, ein logischer Wert `TRUE/FALSE`, der angibt, ob der Artikel über dem Ressort-Median liegt

Wie viele Artikel im Ressort `"inland"` liegen über dem eigenen Ressort-Median? Ist das überraschend?

24. Nach `group_by()` `|> mutate()` bleibt der Datensatz in R *weiterhin gruppiert*. Das kann zu unerwarteten Ergebnissen führen, wenn ihr den Datensatz danach ohne explizites `ungroup()` weiterverwendet.

Zeigt diesen Effekt anhand eines konkreten Beispiels: Erstellt einen nach Ressort gruppierten Datensatz mit einer neuen Spalte `n_ressort = n()`. Ruft darauf `slice_max(word_count, n = 1)` auf, *ohne* vorher `ungroup()` zu verwenden. Was erhaltet ihr? Ruft dann `ungroup()` auf und wiederholt `slice_max()`. Was ändert sich? Wann muss man `ungroup()` explizit aufrufen?

3.5. Schritt 5: Datensätze verknüpfen mit Joins

In der Praxis liegen Informationen selten in einer einzigen Tabelle vor. *Joins* ermöglichen es, Tabellen über gemeinsame Schlüsselspalten zu verknüpfen. In diesem Teil erstellt ihr eine

Metadaten-Tabelle und verknüpft sie auf verschiedene Weisen mit dem Hauptdatensatz.

25. Erstellt in R eine kleine Metadaten-Tabelle `ressort_meta` mit folgenden Spalten:

- `ressort`, der Ressort-Schlüssel, wie er im Datensatz vorkommt, zum Beispiel "inland"
- `ressort_label`, ein leserfreundlicher Anzeigename, zum Beispiel "Inland"
- `themenbereich`, eine übergeordnete thematische Kategorie, zum Beispiel "Politik", "Wirtschaft", "Gesellschaft" oder "Faktencheck"

Befüllt die Tabelle mit Einträgen für mindestens die acht häufigsten Ressorts. Lasst dabei bewusst einige seltene Ressorts *weg*, das wird in den nächsten Aufgaben relevant.

26. Verknüpft den auf die Hauptvariablen reduzierten Datensatz mit `ressort_meta` über einen `left_join()`, verbunden über die gemeinsame Spalte `ressort`. Was bedeutet ein `left_join()` in diesem Kontext, welche Zeilen bleiben garantiert erhalten?

Prüft das Ergebnis. Für wie viele Artikel wurde kein passender Eintrag in `ressort_meta` gefunden, erkennbar an `NA` in `ressort_label`? Welche Ressorts sind das?

Erstellt anschließend ein Balkendiagramm, das die Anzahl der Artikel pro `themenbereich` zeigt, sortiert nach Häufigkeit. Behandelt Artikel ohne Match als eigene Kategorie "Nicht klassifiziert", Tipp: `replace_na()`, und hebt diese Kategorie farblich ab.

27. Führt denselben Join als `inner_join()` durch. Was ist der Unterschied zum `left_join()` aus Aufgabe 26? Wie viele Zeilen hat das Ergebnis im Vergleich?

Erstellt mit dem Ergebnis des `inner_join()` einen horizontalen Boxplot der Wortanzahl, gruppiert nach `themenbereich`, gefiltert auf `word_count < 3000`, sortiert nach Median. Welcher Themenbereich produziert die längsten Artikel?

28. Verwendet `anti_join()`, um herauszufinden, welche Artikel im Hauptdatensatz einen Ressort-Wert haben, der *nicht* in `ressort_meta` vorkommt. Gebt die fehlenden Ressorts mit ihrer Häufigkeit aus, sortiert nach Häufigkeit.

Was ist der praktische Nutzen von `anti_join()` beim Arbeiten mit Lookup-Tabellen? Wie hängt der `anti_join()` mit dem `left_join()` zusammen? Was ist der Bezug zwischen den Zeilen, die beim `left_join()` den Wert `NA` erhalten, und dem Ergebnis des `anti_join()`?

Erstellt ein Balkendiagramm der nicht gematchten Ressorts, nur solche mit mindestens 5 Artikeln, ohne `NA`. Hebt mit einer abweichenden Farbe die Ressorts hervor, die ihr für eine vollständigere Lookup-Tabelle als prioritär erachtet, zum Beispiel weil sie mehr als 100 Artikel enthalten.

4. Daten visualisieren

Am Beispiel der Tagesschau-Daten

Eine gute Visualisierung macht sichtbar, was in Tabellen und Kennzahlen verborgen bleibt. Aber Visualisierungen können auch täuschen: durch abgeschnittene Achsen, ungeschickte Filterentscheidungen oder suggestive Skalierungen. In dieser Übung lernt ihr zunächst die wichtigsten Diagrammtypen für Häufigkeiten, Gruppenvergleiche, Zeitreihen und Zusammenhänge kennen. Anschließend setzt ihr euch gezielt mit typischen Fallstricken auseinander, die auch erfahrenen Analytikerinnen und Analytikern unterlaufen.

4.1. Schritt 1: Daten laden und vorbereiten

Diese Übung baut auf der vorherigen Übung zur Datentransformation auf. Stellt sicher, dass euer Projekt bereits eingerichtet ist und der Datensatz in `data/` liegt.

1. Ladet die benötigten Pakete mit `pacman::p_load(): tidyverse, ggribes und scales`. Lest anschließend den Datensatz ein und erstellt in einem Schritt alle abgeleiteten Variablen, die ihr für diese Übung benötigt: `year`, `month`, `weekday` (mit `lubridate::wday(..., label = TRUE)`) sowie `title_length` (Zeichenanzahl des Titels). Erstellt außerdem den reduzierten Datensatz `news_slim` und den aggregierten `news_timeline`.

4.2. Schritt 2: Häufigkeiten und Verteilungen

Die grundlegenden Visualisierungsformen für einzelne Variablen sind das Balkendiagramm für kategoriale und das Histogramm für metrische Variablen.

2. Erstellt ein Balkendiagramm, das die Häufigkeitsverteilung der Ressorts zeigt. Filtert vorab auf die sechs Hauptressorts ("`inland`", "`ausland`", "`wirtschaft`", "`wissen`", "`investigativ`", "`faktenfinder`"). Die Balken sollen nach Häufigkeit *absteigend* sortiert sein.

Beschriftet beide Achsen sinnvoll, wählt einen aussagekräftigen Diagrammtitel und verwendet ein schlichtes Theme.

3. Erstellt ein Histogramm der Variable `word_count`. Filtert zuvor Artikel mit mehr als 3.000 Wörtern heraus, damit die Verteilung übersichtlich bleibt. Experimentiert mit verschiedenen `bins`-Werten, zum Beispiel 30, 60 und 100, um eine aussagekräftige Klasseneinteilung zu finden.

Fügt dem Histogramm mit `geom_vline()` eine vertikale Linie für den Median hinzu. Beschreibt die Verteilung. Ist sie symmetrisch, links- oder rechtsschief? Was sagt das über typische Artikellängen auf tagesschau.de aus?

4. Untersucht, welche Tags am häufigsten vergeben wurden. Filtert zunächst alle Tags heraus, die mindestens 200 Mal vorkommen, und entfernt `NA`-Werte.

Achtung: Die Tags `"FAKTENFINDER"` und `"#FAKTENFINDER"` bezeichnen dasselbe Konzept. Bereinigt das mit `str_remove_all()`, bevor ihr zählt.

Stellt die Top-Tags als sortiertes Balkendiagramm dar, absteigend nach Häufigkeit. Welche thematischen Schwerpunkte lassen sich aus der Tag-Verteilung ablesen?

4.3. Schritt 3: Gruppenvergleiche visualisieren

Nicht nur einzelne Variablen, sondern Unterschiede zwischen Gruppen sind oft am aufschlussreichsten. Boxplots, Violinplots und Ridgeline-Diagramme eignen sich dafür besonders gut. Heatmaps helfen, zwei kategoriale Dimensionen gleichzeitig zu erkunden.

5. Erstellt einen Boxplot, der die Verteilung der Wortanzahl für die sechs Hauptressorts zeigt. Filtert Artikel mit mehr als 3.000 Wörtern heraus. Sortiert die Ressorts nach ihrem Median, Tipp: `fct_reorder(ressort, word_count, .fun = median)`, sodass das Ressort mit dem niedrigsten Median oben steht. Verwendet `coord_flip()` für eine horizontale Darstellung.

Welches Ressort hat die größte Streuung? Welches die kleinste?

6. Ergänzt den Boxplot aus Aufgabe 5 um überlagerte Datenpunkte. Setzt `alpha = 0.1` und `size = 0.5`, um Überlappungen zu reduzieren. Achtet auf die Reihenfolge der `geom_`-Aufrufe. Der Boxplot sollte *über* den Punkten liegen, nicht darunter.

Ab welcher Datenmenge gerät diese Darstellung an ihre Grenzen? Was ist der Vorteil dieser kombinierten Form gegenüber einem reinen Boxplot? Schaut euch mal `geom_sina` aus dem `ggforce`-Paket an.

7. Installiert das Paket `ggridges` und erstellt mit `geom_density_ridges()` ein Ridgeline-Diagramm der Wortanzahl-Verteilung für alle Hauptressorts. Die Filterung auf `word_count < 3000` bleibt bestehen. Füllt die Flächen mit `fill = ressort` und wählt eine ansprechende Farbpalette.

Beschreibt den Unterschied zwischen einem Boxplot und einem Ridgeline-Diagramm. Welche Informationen zeigt jede Darstellungsform, welche verbirgt sie? Wann würdet ihr welche Form bevorzugen?

8. Erstellt eine Heatmap, die zeigt, wie viele Artikel an welchem Wochentag in welchem Hauptressort erschienen. Berechnet dafür zunächst die Häufigkeiten pro Wochentag und Ressort mit `group_by()` und `summarize()`. Visualisiert das Ergebnis mit `geom_tile()` und `scale_fill_viridis_c()`.

Zeigt die Wochentage auf der x-Achse, Montag bis Sonntag, die Ressorts auf der y-Achse. Was fällt euch beim Wochenendverhalten auf? Unterscheiden sich die Ressorts darin?

4.4. Schritt 4: Zeitliche Entwicklungen visualisieren

Der Tagesschau-Datensatz erstreckt sich über fast zwei Jahrzehnte. Zeitreihenvisualisierungen helfen uns, Trends, Wachstum und saisonale Muster zu erkennen.

9. Erstellt ein Liniendiagramm, das die Gesamtanzahl der Artikel pro Jahr zeigt. Markiert die Datenpunkte zusätzlich mit `geom_point()`. Beschriftet die y-Achse sinnvoll und dreht die x-Achsenbeschriftungen, falls nötig.

Welche Jahre stechen besonders hervor? Versucht, den starken Anstieg ab 2023 inhaltlich zu erklären. Berücksichtigt dabei, dass 2006 und 2026 möglicherweise keine vollständigen Jahrgänge sind. Wie könnte man diesen Sachverhalt im Diagramm kenntlich machen?

10. Erstellt ein Liniendiagramm, das die Artikelanzahl pro Jahr *für die drei Ressorts "inland", "ausland" und "wirtschaft" gleichzeitig* als drei farbige Linien zeigt. Fügt eine Legende hinzu und wählt aussagekräftige Farben mit `scale_color_manual()` oder einer vorgefertigten Palette.

Welches Ressort ist in den letzten Jahren am stärksten gewachsen? Gibt es Phasen, in denen ein Ressort vorübergehend besonders aktiv war?

11. Erstellt ein gestapeltes Flächendiagramm, `geom_area()`, der Artikelanzahl pro Jahr für die sechs Hauptressorts. Wählt `position = "stack"`, also gestapelt. Vergleicht die Darstellung mit dem Liniendiagramm aus Aufgabe 10.

Was lässt sich aus der Gesamtfläche ablesen, was im Liniendiagramm nicht sichtbar war? Welche Darstellungsform ist besser geeignet, um den Anteil einzelner Ressorts am Gesamtvolumen zu zeigen?

12. Erstellt eine Heatmap, die zeigt, wie viele Artikel pro Jahr *und Monat* erschienen. Die x-Achse soll die Monate zeigen, Januar bis Dezember, die y-Achse die Jahre, mit dem neuesten Jahr oben, Tipp: Faktor mit umgekehrter Reihenfolge. Verwendet `geom_tile()` und `scale_fill_viridis_c(option = "plasma")`.

Lassen sich saisonale Muster erkennen? Ab welchem Jahr nimmt die Datendichte deutlich zu? Was könnte das über die Entstehungsgeschichte des Datensatzes verraten?

4.5. Schritt 5: Zusammenhänge und mehrdimensionale Analysen

Scatter-Plots sind das klassische Werkzeug, um Zusammenhänge zwischen zwei metrischen Variablen zu erkunden. Wenn eine dritte Variable ins Spiel kommt, setzen wir Farbe oder Facettierung ein.

13. Gibt es einen Zusammenhang zwischen der Länge eines Titels `title_length` und der Länge des zugehörigen Artikels `word_count`? Zieht zunächst mit `slice_sample(n = 3000)` ein Zufallssample, um die Darstellung übersichtlich zu halten. Erstellt dann einen Scatter-Plot der beiden Variablen und fügt mit `geom_smooth(method = "lm")` eine lineare Trendlinie hinzu.

Beschreibt Stärke und Richtung des sichtbaren Zusammenhangs. Überprüft euren visuellen Eindruck mit `cor()`. Wie stark ist die Korrelation tatsächlich? Ist das Ergebnis überraschend, und warum oder warum nicht?

14. Erweitert den Scatter-Plot aus Aufgabe 13 um eine dritte Dimension. Färbt die Punkte nach `ressort` ein, nur die sechs Hauptressorts. Verwendet anschließend `facet_wrap(~ressort)`, um für jedes Ressort ein eigenes Teildiagramm zu erzeugen.

Unterscheidet sich der Zusammenhang zwischen Titellänge und Artikellänge zwischen den Ressorts?

15. Untersucht den Tagesrhythmus der Redaktion. Berechnet für jede Stunde des Tages, 0 bis 23 Uhr, die Anzahl der veröffentlichten Artikel. Visualisiert das Ergebnis als Flächen- oder Balkendiagramm.

Wann ist die Redaktion am produktivsten? Wann ist die Aktivität am niedrigsten? Gibt es Muster, die auf einen typischen Redaktionsalltag hindeuten?

4.6. Schritt 6: Fallstricke bei Visualisierungen

Visualisierungen lügen selten — aber sie täuschen oft. Und meistens liegt das nicht an böser Absicht, sondern an unüberlegten Standardentscheidungen: einer Achse, die nicht bei Null beginnt, einem Filter, der stillschweigend Datenpunkte entfernt, oder einer Skalierung, die Trends dramatischer wirken lässt als sie sind. In diesem Schritt reproduziert ihr typische Fehler absichtlich und korrigiert sie anschließend.

16. Erstellt zwei Versionen desselben Balkendiagramms der Ressort-Häufigkeiten (Hauptressorts, sortiert nach Häufigkeit):

- Version A: Die y-Achse beginnt bei 15.000 (z.B. mit `coord_cartesian(ylim = c(15000, NA))`).
- Version B: Die y-Achse beginnt bei 0.

Beschreibt, wie sich der visuelle Eindruck unterscheidet. Welches Ressort wirkt in Version A wie viel Mal häufiger als ein anderes, obwohl der tatsächliche Unterschied viel kleiner ist?

Edward Tufte hat dafür das Prinzip des *Proportional Ink* formuliert: Die Menge an Tinte (oder Pixeln), die eine Datengröße repräsentiert, soll proportional zu dieser Größe sein. Erklärt in eigenen Worten, warum Version A dieses Prinzip verletzt. Wann ist es dagegen akzeptabel, die Achse nicht bei 0 beginnen zu lassen?

17. Ihr habt in Aufgabe 3 den Filter `word_count < 3.000` angewendet, bevor ihr das Histogramm gezeichnet habt. Wie stark beeinflusst dieser Filter das Bild?

Erstellt zwei Boxplots der Wortanzahl für die Hauptressorts nebeneinander: einen ohne Filter und einen mit dem Filter `word_count < 3000`. Berechnet außerdem, wie viel Prozent der Artikel pro Ressort durch den Filter entfernt werden.

Verändert der Filter Median oder IQR nennenswert? Ändert er die *Interpretation* der Ressortunterschiede? Was sollte man dokumentieren, wenn man einen solchen Filter anwendet?

18. Ein Boxplot zeigt Median, Quartile und Ausreißer, aber er verbirgt die Form der Verteilung. Stellt die Wortanzahl für das Ressort "ausland" auf drei Arten dar: als Histogramm, als Boxplot und als Violinplot (`geom_violin()`). Filtert jeweils auf `word_count < 3000`.

Was sieht man im Histogramm, das der Boxplot nicht zeigt? Was leistet der Violinplot im Vergleich? In welchen Situationen würdet ihr trotzdem einen reinen Boxplot wählen?

19. Zeitreihen reagieren besonders empfindlich auf die Wahl der Achsenskalierung. Nehmt die jährliche Artikelanzahl für die Jahre 2015 bis 2025 und erstellt zwei Liniendiagramme:

- Version A: y-Achse beginnt bei 0.
- Version B: y-Achse zeigt nur den Bereich der tatsächlichen Datenwerte, z.B. mit `coord_cartesian(ylim = c(2000, NA))`.

Welche Version lässt den Wachstumstrend dramatischer wirken? Ist Version B "falsch"? Wann ist es bei Liniendiagrammen vertretbar, die y-Achse nicht bei 0 zu beginnen? Wie könnte man Leser darauf hinweisen, dass die Achse ausgeschnitten ist?

20. Doppelte y-Achsen (*dual axes*) sind verlockend, wenn zwei Zeitreihen sehr unterschiedliche Skalen haben. Versucht, die jährliche Artikelanzahl für "inland" und "ausland" in einem einzigen Diagramm mit zwei y-Achsen darzustellen. In ggplot2 geht das über `sec_axis()` in `scale_y_continuous()`.

Warum ist diese Darstellung grundsätzlich problematisch? Konstruiert ein Beispiel, in dem die visuelle Aussage über den Zusammenhang beider Reihen durch die Wahl des Skalierungsfaktors umgekehrt werden kann. Erstellt dann ein *Small Multiples*-Diagramm mit `facet_wrap()` als sauberere Alternative und vergleicht beide.

4.7. Schritt 7: Reflexion

21. Erklärt in eigenen Worten den Unterschied zwischen `summarize()` und `mutate()` in Kombination mit `group_by()`. Nennt je ein konkretes Beispiel aus dieser Übung, bei dem ihr lieber `summarize()` und eines, bei dem ihr lieber `group_by() + mutate()` eingesetzt habt, und begründet eure Wahl.

22. Ihr habt in dieser Übung fast zwei Jahrzehnte Tagesschau-Berichterstattung analysiert. Welche drei Beobachtungen haben euch am meisten überrascht oder sind euch besonders aufgefallen? Versucht, jede Beobachtung auch inhaltlich, über das bloße Zahlenmuster hinaus, zu erklären oder einzuordnen.

23. Betrachtet alle Visualisierungsformen aus dieser Übung: Histogramm, Balkendiagramm, Boxplot, Boxplot mit Punktwolke, Violinplot, Ridgeline-Plot, Heatmap, Liniendiagramm, Flächendiagramm und Scatter-Plot. Erstellt eine eigene gedankliche Klassifikation. Welche Form ist für welche Kombination aus Fragestellung (univariat, bivariat, trivariat) und Variablentyp (metrisch, kategorial, zeitlich) am besten geeignet? Gibt es Überschneidungen, in denen mehrere Formen in Frage kämen? Was bestimmt dann die Wahl?

24. Datenqualität ist ein unterschätztes, aber entscheidendes Thema. Nennt mindestens vier konkrete Hinweise auf potenzielle Qualitätsprobleme oder Einschränkungen im Tagesschau-Datensatz, die euch im Laufe dieser Übung aufgefallen sind. Denkt dabei an fehlende Werte, uneinheitliche Kodierungen, unvollständige Jahrgänge, Veränderungen der Datenquelle über die Zeit und mögliche Selektionseffekte. Wie würdet ihr diese Probleme in einer veröffentlichungswürdigen Analyse dokumentieren und behandeln?

25. In dieser Übung haben wir ausschließlich mit den strukturierten Merkmalen im Datensatz gearbeitet. Wie könnten wir die unstrukturierten Merkmale wie den Artikeltext oder die Titel in unsere Analysen einbeziehen? Was müssten wir vorher tun?

5. Regelbasierte Textanalyse

Am Beispiel der Tagesschau-Daten

Text in Nachrichtenartikeln ist unstrukturiert, aber nicht formlos. Überschriften folgen Mustern. Autoren wiederholen sich. Themen tauchen auf, verschwinden und kehren wieder. Regelbasierte Textanalyse macht genau diese Muster sichtbar, indem sie auf rohen Text Struktur projiziert: durch Suchen, Extrahieren, Transformieren und Zählen. In dieser Übung baut ihr mit dem Tagesschau-Datensatz eine vollständige Analysepipeline auf, vom einfachen Keyword-Suchen bis zur dictionary-basierten Themenklassifikation über zwei Jahrzehnte Nachrichtengeschichte.

5.1. Schritt 1: Pakete laden und Daten einlesen

1. Installiert und ladet mit `pacman::p_load()` die folgenden Pakete: `tidyverse`, `skmr`, `tidytext`, `jsonlite`, `stopwords`. Beschreibt in einem Satz, wozu `tidytext`, `jsonlite` und `stopwords` jeweils eingesetzt werden.

2. Lest den Tagesschau-Datensatz mit `read_csv()` ein. Ergänzt direkt im selben Pipeline-Schritt zwei neue Variablen: `year` (Erscheinungsjahr aus `date_time` mit `lubridate::year()`) und `date` (nur das Datum als `as.Date(date_time)`).

Welche Spalten des Datensatzes enthalten freien Text, welche strukturierte Werte?

5.2. Schritt 2: Texte durchsuchen und klassifizieren

3. Nutzt `str_detect()`, um vier neue logische Spalten zu erzeugen:

- `thema_klima` (Muster "Klima"),
- `thema_corona` (Muster "Corona|COVID"),
- `thema_ukraine` (Muster "Ukraine") und
- `thema_ki` (Muster "\bKI\b|[Kk]ünstliche Intelligenz").

Sucht jeweils in der **title**-Spalte. Wie viele Artikel enthalten pro Thema das Muster?

4. Stellt die zeitliche Entwicklung aller vier Themen in einem Liniendiagramm dar. Berechnet zunächst die Anzahl der Treffer pro Jahr und Thema, überführt das Ergebnis mit `pivot_longer()` in Langform und plottet es mit `ggplot2`. Wann war Corona am stärksten präsent? Wann beginnt das Thema KI merklich zu wachsen?

5. Die absoluten Zählungen aus Aufgabe 4 können irreführend sein: In späteren Jahren erscheinen generell mehr Artikel, sodass ein absoluter Anstieg auch einfach das gestiegene Gesamtvolumen widerspiegeln kann, statt eines echten inhaltlichen Bedeutungsgewinns. Berechnet daher die *relative Häufigkeit* jedes Themas pro Jahr als Anteil aller Artikel des jeweiligen Jahres in Prozent. Stellt die relativen Frequenzen als Liniendiagramm dar.

Vergleicht das Ergebnis mit dem Diagramm aus Aufgabe 4. Bei welchen Themen verändert sich das Bild? Was lässt sich nur aus der relativen Darstellung schließen?

6. `stringr` bietet neben `str_detect()` auch `str_starts()` und `str_ends()`. Nutzt `str_starts()`, um Artikel zu finden, deren Titel mit "Interview:" oder mit "Liveblog:" beginnen. Nutzt `str_ends()`, um Titel zu finden, die mit einem Fragezeichen enden. Wie viele Treffer liefern die drei Muster jeweils? Was ist der Unterschied zu `str_detect()` für dasselbe Suchmuster?

7. Viele Tagesschau-Titel folgen dem Muster "Format: Überschrift". Erstellt eine neue Variable `format`, die den Artikeltyp klassifiziert. Nutzt `case_when()` in Kombination mit `str_starts()` und Vergleichen auf die `tag`-Spalte. Unterscheidet mindestens: *Interview*, *FAQ*, *Analyse*, *Hintergrund*, *Kommentar*, *Liveblog* und *Meldung* als Auffangkategorie. Wie häufig ist jedes Format?

8. Stellt die Formatverteilung für die vier größten Ressorts (`ausland`, `wirtschaft`, `inland`, `wissen`) als gestapeltes Balkendiagramm mit relativen Anteilen dar. Nutzt dafür `geom_col(position = "fill")` oder berechnet die Anteile explizit mit `mutate(anteil = n / sum(n))`. Welches Ressort hat den höchsten Anteil an FAQ-Artikeln? Welches an Analysen?

5.3. Schritt 3: Texte bereinigen und strukturieren

Reale Datensätze sind selten sauber. Die Tagesschau-Daten enthalten HTML-Fragmente in Autorennamen, fehlerhafte Ressort-Einträge und uneinheitliche Schreibweisen. Bevor ihr analysiert, müsst ihr bereinigen.

9. Schaut euch die `ressort`-Spalte genau an: Gebt alle Ressorts mit weniger als 100 Artikeln aus. Welche Einträge sind offensichtlich fehlerhaft oder technischen Ursprungs? Erstellt anschließend eine bereinigte Version `news_clean`, die nur Artikel mit sinnvollen Ressort-Werten enthält. Wie viele Zeilen werden dabei entfernt?

10. Die Spalte `author` enthält viele Unregelmäßigkeiten: HTML-Tags wie `<em>...</em>`, Präfixe wie "Von " und Quellenangaben wie ", **ARD-Studio Brüssel**". Bereinigt die Spalte schrittweise und speichert das Ergebnis als `author_clean`. Wendet folgende Schritte der Reihe nach an: (1) HTML-Tags entfernen, (2) führendes "Von " entfernen, (3) alles ab dem ersten Komma entfernen, (4) Leerzeichen normalisieren mit `str_squish()`. Überprüft das Ergebnis mit einigen Beispielzeilen.

11. Ermittelt mit der bereinigten `author_clean`-Spalte die 15 produktivsten Autorinnen und Autoren im Datensatz. Filtert leere Strings und `NA`-Werte heraus. Stellt das Ergebnis als horizontales Balkendiagramm dar, sortiert nach Häufigkeit. Welche Redaktionen oder Studios tauchen auf?

12. Viele Tagesschau-Titel folgen dem Muster "Stichwort: Eigentliche Überschrift". Trennt dieses Muster auf: Erstellt eine Variable `stichwort` (Text vor dem ersten Doppelpunkt, bereinigt mit `str_trim()`) und eine Variable `ueberschrift` (Text nach dem Doppelpunkt). Für Artikel ohne Doppelpunkt soll `stichwort` `NA` sein und `ueberschrift` dem vollen Titel entsprechen. Welche zehn Stichworte kommen am häufigsten vor?

13. Reguläre Ausdrücke (*Regex*) ermöglichen komplexere Suchmuster. Sucht in der `title`-Spalte nach:

- (a) Prozentzahlen wie "47 Prozent" oder "3,5 Prozent"
- (b) Zitate in Anführungszeichen

Wie viele Treffer gibt es jeweils? Gebt fünf Beispieltitel pro Kategorie aus.

5.4. Schritt 4: Strukturierte Textfelder parsen

Manche Spalten sind keine einfachen Zeichenketten, sondern serialisierte Datenstrukturen: JSON-Arrays, die als Text gespeichert wurden. `jsonlite::fromJSON()` macht daraus echte R-Objekte, die ihr dann mit den gewohnten tidyverse-Werkzeugen weiterverarbeiten könnt.

14. Die Spalte `keywords` enthält JSON-Arrays wie `["EU", "Trump", "Ukraine"]`. Parst diese Spalte mit `map()` und `jsonlite::fromJSON()` und bringt den Datensatz mit `unnest()` in eine Langform, sodass jede Zeile ein Keyword enthält. Nutzt `possibly()` aus `purrr`, um fehlerhafte JSON-Strings sicher abzufangen. Wie viele Keyword-Einträge gibt es insgesamt? Erstellt eine Rangliste der 20 häufigsten Keywords als Balkendiagramm.

15. Wählt vier Keywords aus (z.B. "Trump", "Coronavirus", "Ukraine", "Klimawandel") und stellt deren Häufigkeit pro Jahr als Liniendiagramm dar. Filtert auf Artikel ab 2010. Lässt sich der Beginn des Ukraine-Kriegs 2022 ablesen? Wann verschwand Coronavirus von der Agenda?

16. Die Spalte `related_links` enthält JSON-Arrays von Objekten. Parst diese Spalte analog zu `keywords`, sodass ihr eine Zeile pro Link habt. Extrahiert aus den Link-URLs mit einem regulären Ausdruck die Domain und erstellt eine Rangliste der 15 häufigsten externen Domains (ohne `tagesschau.de` und `ard.de`).

5.5. Schritt 5: Tokenisierung und Worthäufigkeiten

Für eine systematische Textanalyse zerlegt man Texte in einzelne *Tokens*, die kleinsten bedeutungstragenden Einheiten. Das `tidytext`-Paket fügt sich dabei nahtlos in den Tidyverse-Workflow ein und macht Texte tabellenartig behandelbar.

17. Tokenisiert die `title`-Spalte mit `unnest_tokens()` aus `tidytext`. Behaltet dabei `url`, `year` und `ressort`. Was macht `unnest_tokens()` automatisch mit Groß- und Kleinschreibung und mit Satzzeichen? Wie viele Tokens enthält der gesamte Datensatz?

18. Ladet mit einer deutschen Stoppwortliste und entfernt diese aus den Daten mit `anti_join()`. Filtert zusätzlich alle Tokens heraus, die kürzer als drei Zeichen sind oder ausschließlich aus Ziffern bestehen (`str_detect(word, "^\\d+$")`). Wie viele Tokens bleiben nach der Bereinigung übrig?

19. Ermittelt die 20 häufigsten Wörter in den Tagesschau-Überschriften nach der Bereinigung und stellt sie als horizontales Balkendiagramm dar. Welche Wörter erscheinen, und entspricht das euren Erwartungen? Gibt es Wörter in der Liste, die ihr für inhaltlich wenig aussagekräftig haltet und die noch herausgefiltert werden könnten?

20. Vergleicht die häufigsten Wörter in den vier größten Ressorts (**ausland**, **wirtschaft**, **inland**, **wissen**) in einem facettierten Diagramm mit je zehn Wörtern pro Ressort. Nutzt `reorder_within()` und `scale_x_reordered()` aus `tidytext`, damit die Wörter innerhalb jedes Facetts korrekt sortiert sind. Was verraten die Unterschiede über die inhaltlichen Schwerpunkte der Ressorts?

21. Die reine Häufigkeit bevorzugt Wörter, die in allen Ressorts vorkommen. *TF-IDF* (Term Frequency – Inverse Document Frequency) gewichtet Wörter so, dass charakteristische, ressortspezifische Begriffe hervorgehoben werden. Berechnet TF-IDF mit `bind_tf_idf(word, ressort, n)` aus `tidytext` und stellt die jeweils fünf charakteristischsten Wörter pro Ressort als facetiertes Balkendiagramm dar.

22. Vergleicht die TF-IDF-Ergebnisse mit den reinen Häufigkeiten aus Aufgabe 19 für ein oder zwei Ressorts. Welche Wörter sind in beiden Listen? Welche tauchen nur bei TF-IDF auf? Was sagt euch das über die Stärken und Schwächen beider Maße als Beschreibung von Textinhalten?

5.6. Schritt 6: Wörterbuch-basierte Klassifikation und Reflexion

Bisher habt ihr einzelne Muster untersucht. Jetzt geht es um systematische Klassifikation: Ihr erstellt ein Themen-Dictionary und wendet es auf den gesamten Datensatz an. Das ist *deduktive Inhaltsanalyse* – ihr bringt eure theoretischen Kategorien mit und sucht sie im Text.

23. Erstellt ein Themen-Dictionary in Excel und ladet es als Tibble mit den Spalten `word` (Kleinbuchstaben) und `thema`. Deckt mindestens vier Themen ab: *Klimapolitik*, *Sicherheit & Krieg*, *Wirtschaft & Finanzen* und *Gesundheit*.

Verwendet pro Thema mindestens fünf charakteristische Wörter. Welche Wörter könnten zu Fehlzuordnungen führen?

24. Wendet das Dictionary auf die tokenisierten Titel an. Nutzt `left_join()` zwischen dem bereinigten Token-Datensatz und dem Dictionary. Wie viele Artikel lassen sich den

vier Themen zuordnen?

25. Stellt die Entwicklung aller vier Themen ab 2006 als Liniendiagramm dar. Beschreibt Auffälligkeiten in der zeitlichen Entwicklung der Themen.

26. Ruft zehn zufällige Artikel auf, die dem Thema *Klimapolitik* zugeordnet wurden (`slice_sample(n = 10)`), und inspiziert ihre Titel. Sind die Zuordnungen korrekt? Gebt je ein konkretes Beispiel für einen möglichen *falsch-positiven* Treffer (Artikel wird fälschlich zugeordnet) und einen möglichen *falsch-negativen* Treffer (Artikel müsste zugeordnet werden, wird es aber nicht).

Was könnt ihr tun, um das Ergebnis zu verbessern?

27. Die regelbasierte Textanalyse, die ihr in dieser Übung angewendet habt, ist eine Form der *deduktiven Inhaltsanalyse*. Diskutiert die folgenden Fragen:

- Was sind die zentralen Stärken dieses Ansatzes gegenüber manuellem Durchlesen?
- Was wäre ein *induktiver* Ansatz (z.B. Topic Modeling), und in welcher Situation würdet ihr ihn bevorzugen?
- Nennt zwei Situationen, in denen regelbasierte Textanalyse klar an ihre Grenzen stößt.

6. LLM-gestützte Textanalyse

Am Beispiel der Tagesschau-Daten

Die regelbasierte Textanalyse hat eine entscheidende Schwäche: Ihr müsst im Voraus wissen, wonach ihr sucht. Wörterbücher müssen manuell erstellt werden, Verneinungen bleiben unsichtbar, und kontextuelles Verstehen ist kaum möglich. Große Sprachmodelle (LLMs) versprechen, genau diese Grenzen zu überwinden: Statt Muster zu suchen, lesen und interpretieren sie Text. In diesem Experiment lernt ihr, wie ihr LLMs systematisch für Textanalyseaufgaben einsetzen könnt – und wo ihre eigenen Grenzen liegen. Das Experiment baut direkt auf dem vorherigen Experiment zur regelbasierten Textanalyse auf und stellt denselben Tagesschau-Datensatz in einen neuen methodischen Kontext.

6.1. Schritt 1: LLMs einrichten und testen

Bevor ihr mit der eigentlichen Analyse beginnt, richtet ihr zwei Zugangswege ein: ein lokal laufendes Modell für datenschutzsensitive und kostenfreie Experimente sowie einen Cloud-Zugang für leistungsstärkere Modelle.

1. Ladet [LM Studio](#) herunter und installiert es. Öffnet LM Studio und ladet über die Suchfunktion ein deutschsprachig-kompetentes Modell, zum Beispiel das kleine *Gemma 4 E2B*. Öffnet nach dem Laden die Chat-Oberfläche und stellt dem Modell eine einfache Frage auf Deutsch: „Was ist die Tagesschau?“

Beschreibt in zwei Sätzen: Wie kompetent wirkt das Modell? Gibt es Fehler oder Auffälligkeiten in der Antwort?

2. Startet den lokalen Server in LM Studio über *Local Server* → *Start Server*. Installiert anschließend in eurem Python-Projekt die notwendigen Pakete:

```
pip install openai pandas matplotlib seaborn tqdm scikit-learn
```

Importiert alle Bibliotheken und definiert zwei Variablen: `MODEL_LOCAL` (Modellname aus LM Studio, z.B. "gemma-3-4b-it" – den genauen Namen findet ihr unter *My Models*) und `MODEL_OPENAI` (z.B. "gpt-4o-mini").

3. Erstellt zwei Clients: einen für LM Studio und einen für die OpenAI API. LM Studio stellt eine OpenAI-kompatible API unter `http://localhost:1234/v1` bereit – ihr könnt das `openai`-Paket verwenden und lediglich `base_url` anpassen. Testet beide Verbindungen mit einer einfachen Anfrage: „Nenne mir in einem Satz, worum es bei der Tagesschau geht.“ Gebt die Antworten beider Modelle aus.

Hinweis: Speichert den OpenAI-API-Schlüssel als Umgebungsvariable `OPENAI_API_KEY`, anstatt ihn direkt im Code einzubetten.

4. Vergleicht lokale und Cloud-basierte LLMs anhand von vier Dimensionen. Füllt die folgende Tabelle aus und begründet eure Einschätzungen kurz:

Dimension	Lokal (LM Studio)	Cloud (ChatGPT)
Kosten		
Datenschutz		
Leistungsfähigkeit		
Geschwindigkeit		

Wann würdet ihr welche Variante bevorzugen?

6.2. Schritt 2: Analysecorpus aufbauen

LLM-Analysen sind pro API-Aufruf kostenpflichtig (Cloud) oder zeitintensiv (lokal). Statt den gesamten Tagesschau-Datensatz mit über 60.000 Artikeln zu analysieren, erstellt ihr zunächst einen repräsentativen Teilcorpus.

5. Ladet den Tagesschau-Datensatz in R und bereitet den Analysecorpus vor:

- Lest `tagesschau.zip` ein und ergänzt eine Spalte `year` (Erscheinungsjahr aus `date_time`).
- Filtert auf die sechs häufigsten Ressorts: `ausland`, `wirtschaft`, `inland`, `wissen`, `investigativ`, `faktenfinder`.
- Entfernt Zeilen, in denen `title` oder `text` fehlen.

Wie viele Artikel bleiben nach dem Filtern übrig?

6. Zieht aus dem gefilterten Datensatz eine **stratifizierte Zufallsstichprobe**: 50 Artikel pro Jahr (Stratifizierung nach `year`). Verwendet `set.seed(42)` für Reproduzierbarkeit. Behaltet die Spalten `date_time`, `year`, `title`, `shorttext`, `text`, `ressort` und `url`. Exportiert den Korpus als `data/tagesschau_sample.csv`, damit ihr ihn anschließend in Python laden könnt.

Wie viele Zeilen hat der exportierte Datensatz insgesamt? Welche Jahre sind enthalten?

6.3. Schritt 3: Einfache NLP-Aufgaben mit LLMs

Mit dem Korpus könnt ihr jetzt konkrete Analyseaufgaben durchführen. Ihr beginnt mit einfacheren Aufgaben, die ihr aus der regelbasierten Analyse kennt – um zu sehen, was LLMs besser, schlechter oder anders machen.

7. Ladet den Korpus in Python und schreibt zwei Hilfsfunktionen:

- `llm_text(prompt, client, model)`: Schickt einen Prompt, gibt die Textantwort zurück.
- `llm_json(prompt, client, model)`: Wie `llm_text`, aber mit `response_format={"type": "json_object"}` – gibt ein Python-Dictionary zurück.

Beide Funktionen sollen `temperature=0` verwenden, damit die Ergebnisse möglichst reproduzierbar sind. Testet beide Funktionen mit einem kurzen Beispielaufruf.

8. Wählt fünf zufällige Artikel aus dem Korpus und lasst das lokale Modell und ChatGPT jeweils eine **Zusammenfassung in zwei deutschen Sätzen** erstellen. Nutzt den `text`-Wert als Eingabe, begrenzt ihn aber auf maximal 600 Zeichen (`text[:600]`), um die Promptlänge zu kontrollieren.

Gebt Titel, Zusammenfassung des lokalen Modells und Zusammenfassung von ChatGPT nebeneinander aus. Wo gibt es inhaltliche Unterschiede oder Fehler?

9. Wie reißerisch ist ein Nachrichtentitel? Schreibt eine Funktion `rate_sensational(title, client, model)`, die einen Artikel-Titel in eine von vier **ordinalen Kategorien** einordnet und den Kategorienamen als String zurückgibt:

- `"neutral"` – sachliche Meldung, keine besondere Wertung

- "pointed" – zugespitzte oder deutlich pointierte Formulierung
- "alarming" – alarmierender oder aufwühlender Ton
- "sensational" – reißerisch, provozierend, Clickbait-Charakter

Wendet die Funktion auf 20 zufällige Artikel an. Gebt alle Titel der Kategorien "alarming" und "sensational" aus. Sind die Einstufungen plausibel?

Außerdem: Warum könnten ordinale Kategorien für LLMs zuverlässiger sein als eine numerische Skala von 1 bis 5?

10. Schreibt eine Funktion `rate_sentiment(title, text, client, model)`, die bewertet, ob ein Artikel eine **gute oder schlechte Nachricht** enthält. Verwendet eine viergliedrige Skala ohne neutrale Mittelkategorie, um das Modell zu einer Entscheidung zu zwingen:

- -2 – sehr schlechte Nachricht (Katastrophe, Tod, schwere Krise)
- -1 – eher schlechte Nachricht (Konflikt, Problem, Rückschlag)
- +1 – eher gute Nachricht (positive Entwicklung, Fortschritt)
- +2 – sehr gute Nachricht (Durchbruch, gelöstes Problem, Erfolg)

Wendet die Funktion auf 30 zufällige Artikel an. Berechnet den Mittelwert pro Ressort. Welches Ressort berichtet am meisten über schlechte Nachrichten?

6.4. Schritt 4: Ressort-Klassifikation und Evaluation

In der regelbasierten Textanalyse habt ihr Artikel über Wörterbücher klassifiziert. Jetzt fragt ihr ein LLM: Kann es das besser – und wie viel besser?

11. Schreibt eine Funktion `predict_section(title, shorttext, client, model)`, die das Ressort eines Artikels vorhersagt. Das Modell soll genau eines der sechs Ressorts zurückgeben: **ausland, wirtschaft, inland, wissen, investigativ, faktenfinder**. Nennt im Prompt die vollständige Liste und fordert eine exakte Antwort ohne weitere Erklärung.

Testet die Funktion an drei Beispielen aus dem Korpus und vergleicht Vorhersage und wahres Label.

12. Wendet die Klassifikation auf einen stratifizierten Test-Datensatz von 60 Artikeln an (10 pro Ressort). Führt die Klassifikation mit beiden Modellen durch (lokal und ChatGPT) und speichert die Ergebnisse in den Spalten `pred_local` und `pred_openai`. Berechnet anschließend die **Genauigkeit** (Anteil korrekt klassifizierter Artikel) für beide Modelle.

Welches Modell schneidet besser ab? Ist der Unterschied größer als erwartet?

13. Visualisiert die Klassifikationsergebnisse als **Konfusionsmatrix** – je eine Heatmap für das lokale Modell und für ChatGPT. Auf der x-Achse stehen die vorhergesagten Ressorts, auf der y-Achse die wahren Ressorts.

Welche Ressorts werden am häufigsten verwechselt? Lässt sich ein inhaltliches Muster erkennen?

14. Berechnet für jedes Ressort **Precision**, **Recall** und **F1-Score** mithilfe von `sklearn.metrics.classification_report`. Gebt den Report für beide Modelle aus.

Welches Ressort ist am schwersten zu klassifizieren? Diskutiert, warum das so ist.

6.5. Schritt 5: Sentiment-Analyse im Zeitverlauf

Mit den Grundlagen aus Schritt 3 könnt ihr jetzt eine umfangreichere Analyse aufsetzen: Wie verändert sich die Grundstimmung der Tagesschau-Berichterstattung über Ressorts und Jahre hinweg?

15. Wendet die Funktion `rate_sentiment()` aus Aufgabe 10 auf den **gesamten Stichprobencorpus** an. Nutzt `tqdm.pandas()` und `progress_apply()`, um den Fortschritt zu verfolgen. Speichert die Ergebnisse als neue Spalte **sentiment** im DataFrame.

Wie viele Werte konnten nicht geparkt werden (d.h. ergaben **None**)? Was könnte der Grund dafür sein?

16. Berechnet den Durchschnittssentiment pro **Ressort** und **Jahr** für Artikel ab 2010. Speichert das Ergebnis als Pivot-Tabelle (Zeilen = Ressorts, Spalten = Jahre). Gebt die Tabelle aus.

17. Visualisiert die Pivot-Tabelle als **Heatmap** (x-Achse: Jahr, y-Achse: Ressort, Farbe: Durchschnittssentiment). Verwendet eine divergierende Farbskala, die negative Werte (rot) von positiven (grün) unterscheidet, mit 0 als Mittelpunkt.

Was fällt auf? In welchen Jahren und Ressorts überwiegt negatives Sentiment? Lassen sich gesellschaftliche Ereignisse (z.B. Wirtschaftskrisen, Pandemie) in der Heatmap ablesen?

6.6. Schritt 6: Wer spricht über wen?

Named-Entity-Recognition (NER) identifiziert Personen, Orte und Organisationen in Texten. Im Unterschied zur regelbasierten Erkennung mit Namenslisten versteht ein LLM den Kontext, erkennt verschiedene Schreibweisen und unterscheidet Personen mit gleichem Nachnamen.

18. Schreibt eine Funktion `extract_persons(text, client, model)`, die alle im Text genannten Personen als Python-Liste zurückgibt. Der Prompt soll das Modell anweisen, ausschließlich Personennamen (Vor- und Nachname) zurückzugeben – keine Orte oder Organisationen. Das Ergebnis soll ein JSON-Dictionary der Form `{"personen": ["Name1", "Name2"]}` sein.

Testet die Funktion an fünf Artikeln und beurteilt die Qualität der Extraktion.

19. Wendet `extract_persons()` auf alle Artikel im Korpus an und bringt das Ergebnis mit `explode()` in Langform: eine Zeile pro genannter Person. Behaltet `year`, `ressort` und `url` als Kontextspalten.

Erstellt eine Rangliste der 20 meistgenannten Personen im gesamten Datensatz. Wer steht an der Spitze, und entspricht das euren Erwartungen?

20. Wählt fünf bekannte Politiker aus dem Datensatz – zum Beispiel *Merkel*, *Scholz*, *Merz*, *Baerbock* und *Habeck*. Schreibt eine Funktion `person_sentiment(text, person, client, model)`, die bewertet, wie die genannte Person in einem Artikel dargestellt wird (Skala -2 bis +2). Gibt `None` zurück, wenn die Person nicht erwähnt wird.

Filtert den `persons_df` auf Artikel, in denen mindestens einer der fünf Politiker vorkommt, und wendet die Funktion an.

21. Aggregiert das Sentiment pro Politiker und Jahr (ab 2010) und visualisiert es als **Heatmap** (x-Achse: Jahr, y-Achse: Politiker, Farbe: Durchschnittssentiment). Verwendet dieselbe divergierende Farbskala wie in Aufgabe 17.

Welche Muster sind erkennbar? Gibt es Jahre, in denen ein Politiker besonders negativ oder positiv dargestellt wurde?

6.7. Schritt 7: Framing-Analyse mit LLMs

Framing beschreibt, welchen Deutungsrahmen ein Artikel einem Thema gibt. Derselbe Migrationsbericht kann als humanitäre Krise, als Sicherheitsbedrohung oder als politischer Streit

gerahmt sein. LLMs können solche impliziten Rahmungen erkennen – etwas, das regelbasierte Verfahren kaum leisten können.

22. Filtert den Korpus auf Artikel zum Thema **Migration**, indem ihr prüft, ob der Titel eines der Wörter „Migration“, „Flüchtling“, „Asyl“, „Migranten“ oder „Einwanderer“ enthält. Wie viele solche Artikel gibt es im Stichprobencorpus?

Schreibt anschließend eine Funktion `analyze_framing(title, text, client, model)`, die das dominante Framing eines Artikels klassifiziert. Verwendet mindestens fünf Kategorien: `humanitaer`, `sicherheitspolitisch`, `rechtlich`, `wirtschaftlich`, `politischer_streit`.

23. Wendet `analyze_framing()` auf alle gefundenen Migrationsartikel an. Erstellt zwei Visualisierungen:

- Ein horizontales Balkendiagramm der Gesamtverteilung aller Framing-Kategorien.
- Ein gestapeltes Balkendiagramm der Framing-Verteilung nach Jahr (falls Artikel aus mehr als zwei Jahren vorliegen).

Welches Framing dominiert in der Tagesschau-Berichterstattung über Migration? Hat sich das im Zeitverlauf verändert?

6.8. Schritt 8: Reflexion

24. Vergleicht die LLM-basierte Analyse mit der regelbasierten Analyse aus dem vorherigen Experiment entlang von vier Dimensionen. Füllt die folgende Tabelle aus und begründet eure Einschätzungen:

Dimension	Regelbasiert	LLM-basiert
Transparenz		
Reproduzierbarkeit		
Skalierbarkeit		
Qualität		

Für welche Aufgaben eignet sich welcher Ansatz besser?

25. Diskutiert die folgenden ethischen und methodischen Fragen der LLM-basierten Inhaltsanalyse:

- Können LLMs politisch voreingenommen sein? Wie würdet ihr das methodisch

testen?

- Welche Risiken entstehen, wenn LLM-basierte Analysen in journalistische oder wissenschaftliche Entscheidungen einfließen?
- Wie lässt sich die Qualität einer LLM-Analyse valide messen – und was bedeutet das für eure Ergebnisse aus diesem Experiment?

Teil III.

Speisepläne

7. Explorative Analyse von Speiseplänen in Deutschlands Mensen

Fallstudie im Sommersemester 2026

Prof. Dr. Nicolas Meseth

Prof. Dr. Melanie Speck

Julia Heinz

7.1. Hintergrund

Die Fallstudie steht im Kontext des [Forschungsprojekts “Linse”](#), in dessen Rahmen Daten zu den Speiseplänen von Mensen an Universitäten und Hochschulen in Deutschland gesammelt wurden. Die Daten stammen aus den IT-Systemen der jeweiligen Studierendenwerke und bieten die Gelegenheit, die Entwicklung der Speisepläne über einen Zeitraum von fast einem Jahrzehnt zu analysieren.

Das Projekt Linse zielt darauf ab, den Leguminosenkonsum zu steigern, und untersucht mögliche Maßnahmen insbesondere im Außer-Haus-Verzehr. Dazu zählen auch Großküchen wie die Mensen der Studierendenwerke, die täglich Tausende von Mahlzeiten zubereiten und damit erheblichen Einfluss auf die Ernährungsgewohnheiten der Studierenden nehmen können.

7.2. Forschungsfragen

Von zentralem Interesse für das Projekt ist, wie sich die Zusammensetzung der Speisepläne in den Mensen über die Zeit verändert hat, insbesondere der Anteil an Hülsenfrüchten und die Hauptproteinquelle der angebotenen Speisen.

Bevor die eigentliche Analyse beginnen kann, müssen die Daten harmonisiert werden: Jede Mensa verwendet eigene Bezeichnungen für die Speisen, viele davon inhaltlich ähnlich oder identisch, und die Rohdaten sind über zahlreiche Excel- und CSV-Dateien mit teils unterschiedlichen Strukturen verteilt.

Auf dieser Grundlage sollen Antworten auf die folgenden Forschungsfragen erarbeitet werden:

- Wie hat sich die Zusammensetzung der Speisepläne in den Mensen über die Zeit entwickelt? Gibt es erkennbare Trends, etwa in Bezug auf die Anzahl der vegetarischen oder veganen Gerichte oder den Anteil von Gerichten mit Hülsenfrüchten? Gibt es Unterschiede zwischen den Mensen und Studierendenwerken?
- Was sind die wichtigsten Proteinquellen und deren Anteile in den angebotenen Gerichten? Wie ist die Entwicklung über die Zeit? Gibt es hier Unterschiede zwischen den Mensen und Studierendenwerken?

7.3. Informationen zum Datensatz

Der Datensatz besteht aus einer Vielzahl von Excel- und CSV-Dateien, die die Speisepläne von insgesamt 31 Mensen von 13 verschiedenen Studierendenwerken in Deutschland über einen Zeitraum von mehreren Jahren enthalten. Insgesamt sind mehr als 1,3 Mio. Einträge enthalten.

Folgende Studierendenwerke sind im Datensatz enthalten:

Studierendenwerk	Dateien	Ordner
Brandenburg Ost	5	data/projekt-linse/brandenburg-ost
Chemnitz-Zwickau	5	data/projekt-linse/chemnitz-zwickau
Erlangen-Nürnberg	1	data/projekt-linse/erlangen-nuernberg
Freiburg	1	data/projekt-linse/freiburg
Kassel	1	data/projekt-linse/kassel
Mainz	1	data/projekt-linse/mainz
Oldenburg	6	data/projekt-linse/oldenburg
Osnabrück	3	data/projekt-linse/osnabrueck
Paderborn	3	data/projekt-linse/paderborn
Saarland	1	data/projekt-linse/saarland
Schleswig-Holstein	1	data/projekt-linse/schleswig-holstein
Stuttgart	1	data/projekt-linse/stuttgart
Wuppertal	2	data/projekt-linse/wuppertal

Jedes Studierendenwerk verwendet eine abweichende Struktur im Hinblick auf die Spaltennamen und die enthaltenen Informationen. Manche Spalten existieren nur in bestimmten Studierendenwerken. Es gibt einige gemeinsame Kernvariablen, die in der folgenden Tabelle zusammengefasst sind. Diese sind mit Ausnahme des Studierendenwerks “Brandenburg Ost” in allen Dateien enthalten. Brandenburg Ost weist eine vollständig abweichende Dateistruktur auf, die nicht mit den übrigen Studierendenwerken kompatibel ist, und wird daher in Teil 1 nicht berücksichtigt:

Variablenname	Beschreibung
Prod.Dat.	Das Datum, an dem die Speise im Plan angeboten wurde
Prod.Art.	Die Art der Speise, wie “Beilage”, “Hauptgericht”, “Dessert” usw.
Bezeichnung	Die vom Studierendenwerk vergebene Bezeichnung der Speise
Produktionsort	Der Ort, an dem die Speise zubereitet wird
Ausgabe	Die Anzahl der Portionen, die tatsächlich ausgegeben wurden
Ausgabetext	Der Text, der auf der Speisekarte erscheint, ggf. mit Zusatzangaben

7.4. Aufgabenstellung

7.4.1. Teil 1: Zusammenführung und Bereinigung

Im ersten Teil dieser Fallstudie überführt ihr die Dateien der Studierendenwerke — mit Ausnahme von Brandenburg Ost, dessen Daten eine vollständig abweichende Struktur aufweisen — in eine einheitliche CSV-Datei. Dafür verwendet ihr R und die Funktionen aus dem Tidymverse. Die Zieltabelle soll die folgende Struktur haben:

Variablenname	Datentyp	Beschreibung
id	dbl	Eine eindeutige fortlaufende ID für jede Zeile
source_file	chr	Der Name der Quelldatei, aus der die Zeile stammt
student_service	fct	Das Studierendenwerk, dem die Zeile zugeordnet werden kann
cafeteria	fct	Die Mensa, der die Zeile zugeordnet werden kann
date	date	Das Datum, an dem die Speise im Plan angeboten wurde
product_type	chr	Die Art der Speise, wie “Beilage”, “Hauptgericht”, “Dessert”
product_name	chr	Die vom Studierendenwerk vergebene Bezeichnung der Speise
production_location	chr	Der Ort, an dem die Speise zubereitet wird
actual_output	dbl	Die Anzahl der Portionen, die tatsächlich ausgegeben wurden
menu_text	chr	Der Text, der auf der Speisekarte erscheint

Speichert die Zieltabelle als `data/menus_consolidated.csv` ab.

Erstellt für die Überführung der Daten ein R-Skript `scripts/consolidate.R`, das von oben nach unten ausführbar ist. Verwendet Kommentare, um die Schritte eurer Datenbereinigung zu

erklären und zu dokumentieren. Beginnt mit dem Laden der Quelldateien, führt dann Schritt für Schritt die notwendigen Transformationen durch und speichert die fertige Zieltabelle als CSV-Datei ab. Nutzt dafür `write_csv()` aus dem `readr`-Paket.

Hinweis

Ein mögliches Ergebnis für den ersten Teil der Fallstudie findet ihr in der Datei `menus_consolidated.csv` im Ordner `data/`. Ihr könnt euch an diesem Ergebnis orientieren und damit in den folgenden Aufgaben arbeiten. Diesen Teil der Fallstudie müsst ihr somit nicht bearbeiten.

7.4.2. Teil 2: Einheitliche Klassifikation der Speisen

Im zweiten Teil klassifiziert ihr die Speisen inhaltlich nach einem Klassifikationsschema, das das Projekt Linse entwickelt hat. Dabei sollt ihr für eine Speise wie folgt vorgehen:

1. Prüft, ob einer Speise eine oder mehrerer Lebensmittelklassen aus dem Klassifikationsschema zugeordnet werden können. Die Zuordnung geschieht auf Basis der (angenommenen) Hauptzutaten einer Speise. Diese können aus dem Speisentext explizit ersichtlich sein, es kann aber auch notwendig sein, sie durch Wissen über die typischen Rezepte eines Gerichts abzuleiten.

Beispiel 1: Ein Gericht “Kalbsschnitzel mit Kartoffeln und Champignonrahmsauce” besteht aus den drei Komponenten “Kalbschnitzel”, “Kartoffeln” und “Champignonrahmsauce”. Eure Klassifikation sollte entsprechend diesem Gericht die Klassen “Rotes Fleisch”, “Knollen / stärkehaltige Gemüse” und “Gemüse” (für die Pilze) sowie “Milchprodukte” (für den Rahm) zuordnen.

Beispiel 2: Ein Gericht “Flammkuchen Griechischer Art” könnte den Lebensmittelklassen “Weizen” (für den Teig), “Milchprodukte” (für den Käse) und “Gemüse” (für die Tomaten) zugeordnet werden. Keine der Komponenten steht explizit im Speisentext.

2. Entscheidet für jede enthaltene Lebensmittelklasse, wie groß der Anteil an der gesamten Speise ist. Verwendet dafür eine einfache qualitative Skala.

Mit der Zuordnung zu einer Lebensmittelklasse ist auch die Zuordnung zu einer Hauptproteinquelle verbunden, die über ein Codesystem definiert ist. Auf der Grundlage der in Schritt 2 ermittelten ungefähren Anteile jeder Klasse könnt ihr später die Proteinzusammensetzung der Speisepläne analysieren und die Entwicklung über die Zeit verfolgen.

Überlegt euch ein effizientes und reproduzierbares Vorgehen, um die Speisen zu Lebensmittelklassen aus dem Klassifikationsschemas zuzuordnen. Nutzt dabei sowohl die Techniken der regelbasierten Arbeit mit Texten, speziell für die Vorverarbeitung, als auch die KI-gestützte

Vorgehensweise mit Sprachmodellen. Auch manuelle Schritte können hilfreich oder notwendig sein.

Es ist wichtig, dass ihr die Genauigkeit eurer Klassifikation bewerten und einordnen könnt. Denkt bei der Entwicklung eures Vorgehens also auch daran, wie er das Ergebnis evaluieren könnt und entscheidet auf dieser Basis, ob eine Veränderung eures Vorgehens das Ergebnis verbessert oder verschlechtert hat.

Eure Verarbeitungsschritte müssen gut dokumentiert, reproduzierbar und nachvollziehbar sein. Erstellt deshalb ein zentrales R-Skript `scripts/classify.R`, von dem aus alle relevanten Schritte ausgeführt werden. Wenn ihr auf Sprachmodelle zurückgreift, ruft die Python-Funktionen über das `reticulate`-Paket und die Funktion `source_python()` auf, damit die gesamte Verarbeitung von einem Skript gesteuert wird. Bei komplexen Schritten mit R könnt ihr den Code in Sub-Skripte auslagern, die aus dem zentralen Skript mittels `source()` aufgerufen werden. Stellt sicher, dass die Skripte fehlerfrei von oben nach unten ausführbar sind und alle Dateiverweise relativ angegeben werden.

Erweitert die Zieltabelle aus Teil 1 um die folgenden Spalten, die die Klassifikation der Speisen enthalten:

Neue Variable	Datentyp	Beschreibung
<code>group_level_1</code>	<code>fct</code>	Die Zuordnung zur Ernährungsform “omnivor”, “vegetarisch” oder “vegan”
<code>group_level_2</code>	<code>fct</code>	Die Zuordnung zur Lebensmittelklasse, etwa “Rotes Fleisch”, “Geflügel” oder “Milchprodukte”
<code>code_main_protein</code>	<code>fct</code>	Die Zuordnung zur Hauptproteinquelle über ein Codesystem

Speichert die erweiterte Zieltabelle mit den obigen Klassifikationsspalten als `menus_classified.csv` im Ordner `data/` ab.

Das Klassifikationsschema für die Lebensmittelgruppen inklusive der Codierung für die Proteinquelle aus dem Projekt Linse findet ihr in der Datei `classification_schema.xlsx` im Ordner `data/projekt-linse`.

 Hinweis: Entwickelt zunächst mit einer Stichprobe

Zieht eine reproduzierbare, zufällige Stichprobe für die Entwicklung eures Vorgehens für die Klassifikation. Für die Evaluation bietet es sich an, die Stichprobe einmal manuell zu klassifizieren und etwa in einer Excel-Datei zu speichern. Damit hättet ihr eine Grundlage für die Evaluation, die dann natürlich automatisch durchgeführt werden sollte.

💡 Hinweis: Gute Vorverarbeitung reduziert den Aufwand

Bei der Einteilung eines Gerichts in Lebensmittelklassen werden euch Sprachmodelle helfen können. Dabei ist es nicht notwendig, alle 1,1 Mio. Einträge durch ein LLM klassifizieren zu lassen. Über eine gute, regelbasierte Vorverarbeitung könnt ihr die Anzahl der Einträge, die ihr an ein LLM übergeben müsst, erheblich reduzieren.

7.4.3. Teil 3: Explorative Analysen

Im dritten Teil der Fallstudie erntet ihr die Früchte der aufwendigen Vorarbeit. Auf der Basis der zusammengeführten, harmonisierten und klassifizierten Daten beantwortet ihr die folgenden Forschungsfragen mit geeigneten Analysen:

- Wie hat sich die Zusammensetzung der Speisepläne in den Mensen über die Zeit entwickelt? Gibt es erkennbare Trends, etwa in Bezug auf die Anzahl der vegetarischen oder veganen Gerichte oder den Anteil von Gerichten mit Hülsenfrüchten? Gibt es Unterschiede zwischen den Mensen und Studierendenwerken?
- Was sind die wichtigsten Proteinquellen und deren Anteile in den angebotenen Gerichten? Wie ist die Entwicklung über die Zeit? Gibt es hier Unterschiede zwischen den Mensen und Studierendenwerken?
- Welche Schlüsse lassen sich zu der Beliebtheit der Speisen mit unterschiedlichen Proteinquellen ziehen? Gibt es hier Unterschiede zwischen den Mensen und Studierendenwerken?

Erstellt ein oder mehrere R-Skripte `scripts/analyze.R` bzw. `scripts/subscripts_analyze/`, in denen ihr sämtliche Analysen durchführt und die Visualisierungen erstellt, die ihr später auf eurem Poster verwendet. Ihr könnt die Analysen auf mehrere Skripte aufteilen, die im zentralen Skript mithilfe von `source()` eingebunden werden.

Das Skript sollte alle Visualisierungen beim Ausführen automatisch erstellen und im Ordner `communications/visualizations/` abspeichern. Verwendet für die Visualisierungen ausschließlich Funktionen aus dem `ggplot2`-Paket (und entsprechend Erweiterungspakete wie z. B. `ggribbles`) und gestaltet sie so, dass sie auf einem wissenschaftlichen Poster präsentiert werden können. Achtet dabei auf eine einheitliche Auswahl der Farben, die Formatierung der Achsen und die Lesbarkeit der Beschriftungen.

7.4.4. Teil 4: Erstellung eines wissenschaftlichen Posters

Auf der Basis eurer Analysen erstellt ihr ein wissenschaftliches Poster, das die wichtigsten Ergebnisse ansprechend und strukturiert präsentiert. Es sollte so klar und übersichtlich gestaltet sein, dass Betrachtende die zentralen Befunde schnell erfassen können.

Erstellt das Poster mit einem Werkzeug eurer Wahl (z. B. PowerPoint, Canva, Inkscape oder Quarto) im Hochformat (Portrait, DIN A0: 841 × 1189 mm) und exportiert es als PDF-Datei. Speichert es im Projekt unter `communications/poster.pdf`. Bringt eine ausgedruckte Version im Format DIN A0 zum Präsentationstermin mit, damit ihr es im Rahmen der Poster-Galerie präsentieren könnt.

7.5. Bewertung

Die Fallstudie wird mit insgesamt 100 Punkten bewertet, die sich wie folgt auf die vier Teile verteilen:

Teil	Gewichtung	Schwerpunkt der Bewertung
Teil 1: Datenzusammenführung	0 %	Vollständigkeit und Korrektheit der Zieltabelle, Codequalität
Teil 2: Klassifikation	50 %	Tiefe und Kreativität der Lösung, Nachvollziehbarkeit des Vorgehens, Reproduzierbarkeit, Abdeckung
Teil 3: Explorative Analysen	30 %	Relevanz der Analysen, Qualität der Visualisierungen
Teil 4: Poster und Präsentation	20 %	Gestaltung, Verständlichkeit, Präsentation und Diskussionsfähigkeit

Übergreifend gelten in allen Teilen folgende Kriterien:

- **Reproduzierbarkeit:** Die Skripte laufen ohne Anpassungen auf einem anderen Rechner durch.
- **Codequalität:** Lesbarer, einheitlich formatierter Code ohne überflüssige Reste oder Kommentare.
- **Dokumentation:** Annahmen und Designentscheidungen sind im Code oder in der `README.md` nachvollziehbar festgehalten.

7.6. Hinweise zur Fallstudie

7.6.1. Bearbeitung

- Beantwortet die Forschungsfragen mithilfe von geeigneten Visualisierungen. Verzichtet auf tabellarische Darstellungen und nutzt sie nur als Ergänzung.
- Führt die explorative Datenanalyse ausschließlich mit R durch. Für sämtliche Datentransformationen und Visualisierungen verwendet ihr Funktionen aus dem Tidyverse.

- Die explorative Datenanalyse lebt von eurer Kreativität und Ausdauer. Gebt euch nicht mit der ersten Analyse zufrieden, die ihr erstellt. Probiert mehrere Ansätze aus. Wenn ihr ein interessantes Muster entdeckt, geht dem tiefer auf den Grund. Obwohl alle Teams die gleichen Aufgaben haben, wird es viele verschiedene Lösungswege geben.
- Wenn ihr bei einer Aufgabe nicht sicher seid, wie sie gemeint ist, oder euch Informationen fehlen, trifft sinnvolle Annahmen und dokumentiert sie. Nutzt die Fragetermine, um Unklarheiten zu beseitigen.
- Lesbarer Code ist besser als schwer verständlicher Code. Verwendet den Pipe-Operator, verteilt lange Ausdrücke auf mehrere Zeilen, rückt den Code ein und fügt Kommentare hinzu, wo es angebracht ist. Entfernt Codereste, die nicht mehr benötigt werden, um die Übersicht zu verbessern.
- Reproduzierbarkeit ist in der Wissenschaft von großer Bedeutung. Stellt sicher, dass eure R-Skripte *ohne zusätzlichen Aufwand* auf einem anderen Computer ausgeführt werden können. Vermeidet also unbedingt absolute Pfadangaben und haltet euch strikt an die Projektstruktur (s. unten).
- Es ist in Ordnung, sich bei der Analyse inspirieren zu lassen. Auch die Übernahme von Ideen oder Code anderer, inklusive der Einsatz von KI, ist erlaubt, solange der verwendete Code verstanden und erklärt werden kann und die Quelle als Kommentar angegeben wird.
- Make it work, make it nice! Erst die Funktion, dann die Ästhetik! Verliert euch nicht in Details wie Achsenformatierung oder Farbpaletten, bevor ihr die korrekte Datenbasis erstellt und die geeignete Visualisierungsform gewählt habt. Eine unpassende Visualisierung, selbst wenn sie optisch ansprechend ist, wird euch nicht helfen. Denkt umgekehrt! Auf eurem Poster sollte aber alles glänzen.

7.6.2. Abgabe

- Die Abgabe erfolgt über eine ZIP-Datei, die alle Dateien enthält, die ihr für die Bearbeitung der Fallstudie verwendet und erstellt habt. Die ZIP-Datei sollte die gleiche Struktur haben wie das bereitgestellte Repository, nur um eure Arbeiten ergänzt:

```
case-study-linse/
├─ data/
│   └─ projekt-linse/
│       ├── brandenburg-ost/
│       ├── chemnitz-zwickau/
│       ├── erlangen-nuernberg/
│       ├── freiburg/
│       ├── kassel/
│       └─ mainz/
```

```

├── oldenburg/
├── osnabrück/
├── paderborn/
├── saarland/
├── schleswig-holstein/
├── stuttgart/
├── wuppertal/
├── menus_consolidated.csv
├── classification_schema.xlsx
├── communications/
├── visualizations/
│   ├── scatter_plot.svg
│   └── ...
├── poster.pdf
├── scripts/
│   ├── subscripts_classify/
│   │   ├── <script_name>.R
│   │   ├── <script_name>.py
│   │   └── ...
│   ├── subscripts_analyze/
│   │   ├── <script_name>.R
│   │   └── ...
│   ├── setup.R
│   ├── classify.R
│   └── analyze.R
└── README.md

```

7.6.3. Postergalerie und Präsentation

- Im Rahmen der Postergalerie präsentiert jede Gruppe ihr Poster in 10 Minuten. Während der Präsentation sollte jedes Gruppenmitglied ungefähr den gleichen Sprechanteil haben.
- Im Anschluss an die Präsentation findet eine Frage- und Diskussionsrunde statt. Stellt sicher, dass jedes Gruppenmitglied auf mögliche Fragen vorbereitet ist. Das gilt auch für das Erläutern bestimmter Schritte eurer Analyse, die vielleicht nicht auf dem Poster dargestellt sind.
- Das Poster im PDF-Format müsst ihr nicht zeitgleich mit den anderen Ergebnissen einreichen. Ihr müsst es aber bis spätestens 21 Uhr am Abend vor der Postergalerie abgegeben haben.

Viel Spaß und Erfolg bei der Bearbeitung der Fallstudie!

Teil IV.

Umfrage

8. Variablen erkunden

Am Beispiel der Marktforschungs-Musterstudie

In dieser Übung arbeitet ihr mit einem Umfragedatensatz aus einer Marktforschungs-Musterstudie zu Schokolade und Milchalternativen. Der Fokus liegt nicht auf komplexen Modellen, sondern auf den Grundlagen guter Datenanalyse: Welche Variablen gibt es, was messen sie, welche Datentypen liegen vor, welche Wertebereiche und Missing-Codes wurden verwendet, und welche Zusammenfassungen und Visualisierungen passen zu unterschiedlichen Variablentypen?

8.1. Schritt 1: Projekt aufsetzen und Daten einlesen

1. Erstellt ein neues Projekt für diese Übung und öffnet es in der Entwicklungsumgebung Positron. Stellt sicher, dass ihr eine *virtuelle R-Umgebung* für das Projekt erstellt und aktiviert habt.

2. Installiert die folgenden Pakete in eurer virtuellen Umgebung: `tidyverse`, `janitor` und `skimr`. Verwendet das Metapaket `pacman`, damit ihr die Pakete mit einem einzigen Befehl installieren und laden könnt.

3. Ladet den Datensatz der Marktforschungs-Musterstudie auf euren Computer herunter und speichert ihn in einem Unterordner `data/` in eurem Projektverzeichnis.

4. Erzeugt einen neuen Ordner `scripts/` in eurem Projektverzeichnis und erstellt eine neue R-Skriptdatei `explore_variables.R` in diesem Ordner.

5. Lest den Datensatz `mds12_schoko_milch.csv` in R ein. Prüft dabei das Dateiformat, zum Beispiel Trennzeichen, Kodierung und Missing-Value-Darstellung. Speichert den

Einlesecode in eurer Skriptdatei und legt direkt ein Objekt an, mit dem ihr in den folgenden Schritten weiterarbeitet.

8.2. Schritt 2: Erster Blick auf den Datensatz

6. Verschafft euch einen ersten Überblick über den Datensatz. Wie viele Beobachtungen und wie viele Variablen sind enthalten? Welche Datentypen wurden beim Einlesen zugewiesen? Welche ersten Hinweise geben euch die Variablennamen über den Aufbau des Fragebogens?

7. Überlegt, wie in diesem Datensatz eine Beobachtung eindeutig identifizierbar gemacht werden kann. Gibt es eine bereits vorhandene Variable, die offensichtlich als Primärschlüssel taugt? Wenn nicht, wie würdet ihr das Problem praktisch lösen?

8. Prüft, welche Informationen ihr allein aus dem Datensatz rekonstruieren könnt und welche ihr nur mit einem Fragebogen oder Codebook sicher interpretieren könnt. Schaut euch insbesondere die Variablennamen für die Fragen 1, 2, 3, 4, 5, 8, 11, 12, 21, 22, 23, 26, 29 und 216 an.

8.3. Schritt 3: Einfache Einzelvariablen analysieren

In diesem Abschnitt betrachtet ihr zunächst einfache Einzelvariablen. Ziel ist, für jede Variable den Typ, den Wertebereich, Missing Values und sinnvolle Zusammenfassungen sauber zu beschreiben.

9. Analysiert die Variablen zu den Fragen 1, 2, 3, 4 und 5, also `q001hheinkauf`, `Q002alter`, `q003land`, `Q004geschlecht` und `q005os`. Prüft jeweils:

- Skalenniveau
- Datentyp in R
- Wertebereich und Füllgrad
- passende Kennzahlen
- eine sinnvolle Visualisierung

Legt für jede dieser Fragen einen kleinen Analyse-Tibble an, der mindestens die `respondent_id` und die jeweils betrachtete Variable enthält.

8.4. Schritt 4: Mehrfachnennungen und Itembatterien

Viele Fragen liegen nicht als einzelne Variable vor, sondern als Block aus mehreren dichotomen Variablen oder als Itembatterie. Diese Struktur verlangt eine andere Vorgehensweise als bei einer einfachen Einzelvariable.

10. Analysiert Frage 8, also den Block `v008ort_*`. Beschreibt zuerst, warum es sich hier nicht um eine einzelne kategoriale Variable, sondern um eine Mehrfachantwort-Struktur handelt. Ermittelt anschließend:

- welche Variablen zum Block gehören
- wie hoch der Füllgrad je Item ist
- wie häufig die einzelnen Antwortoptionen gewählt wurden
- eine geeignete Visualisierung

11. Analysiert die Fragen 11 und 12, also die Blöcke `p011regio_*` und `p012neo_*`. Prüft, welche Codes als echte Antwortkategorien und welche eher als Sonder- oder Missing-Codes zu interpretieren sind. Bereitet die Daten für die Analyse so auf, dass sich die inhaltlichen Bewertungen sinnvoll zusammenfassen und visualisieren lassen.

12. Analysiert die Fragen 21, 22 und 23, also die Blöcke `v021pack_*`, `v022kenn_*` und `v023frei_*`. Identifiziert die Struktur dieser Blöcke und vergleicht, wie häufig einzelne Optionen genannt werden. Welche Visualisierungsform eignet sich, um die wichtigsten Nennungen pro Frageblock übersichtlich zu zeigen?

8.5. Schritt 5: Rangfolgen, Imagebatterien und Mediennutzung

13. Analysiert Frage 26, also die Blöcke `p026gericht1_*` und `p026gericht2_*`. Beschreibt zunächst, was die Struktur dieser Variablenform nahelegt. Vergleicht anschließend die Verteilungen ausgewählter Items und erstellt mindestens eine Visualisierung, die zeigt, wie unterschiedlich die Gerichte bewertet oder eingeordnet wurden.

14. Analysiert Frage 29, also den Block `p029mik_*`. Untersucht, ob es sich um eine Itembatterie handelt, welche Antwortwerte vorkommen und wie stark die einzelnen Aussagen im Mittel ausfallen. Visualisiert das Ergebnis so, dass die wichtigsten Unterschiede zwischen den Aussagen schnell erkennbar sind.

15. Analysiert Frage 216, also den Block `d216medien_*`. Prüft zunächst den Typ dieser Variablenstruktur und beschreibt anschließend, welche Medienkanäle im Datensatz am häufigsten genannt oder genutzt werden. Wählt eine Visualisierung, die die wichtigsten Kanäle klar vergleichbar macht.

8.6. Schritt 6: Reflexion

16. Fasst zusammen, welche grundsätzlichen Variablentypen euch in diesem Datensatz begegnen. Nennt für jeden Typ mindestens ein Beispiel aus der Übung und erläutert kurz, welche Kennzahlen und Visualisierungen jeweils gut dazu passen.

17. Diskutiert, warum Sondercodes wie -1 und -2 in Umfragedaten analytisch problematisch sind, wenn man sie unreflektiert wie normale Werte behandelt. Was würdet ihr in einer dokumentierten Analyse tun, um mit solchen Codes sauber umzugehen?

18. Überlegt abschließend, was euch für eine wirklich publikationsfähige Analyse dieser Variablen noch fehlt. Welche zusätzlichen Informationen aus Fragebogen, Feldbericht oder Codebook würdet ihr unbedingt anfordern, bevor ihr inhaltliche Schlussfolgerungen veröffentlicht?

9. Gruppierte Betrachtung von Variablen

Am Beispiel der Marktforschungs-Musterstudie

In dieser Übung betrachtet ihr Variablen nicht mehr nur einzeln, sondern in Gruppen. Ihr untersucht also, wie sich Häufigkeiten, Mittelwerte und Verteilungen verändern, wenn ihr nach Merkmalen wie Geschlecht, Bundesland oder Altersgruppe aufteilt. Der Schwerpunkt liegt auf `group_by()`, `summarize()`, `mutate()` und darauf, gruppierte Ergebnisse sinnvoll zu visualisieren.

9.1. Schritt 1: Erste Schritte mit Gruppierungen

1. Erstellt ein neues R-Skript in eurem Projekt und öffnet es für die Bearbeitung. Ladet die notwendigen Pakete mittels `p_load` und lest den Datensatz der Musterstudie ein.

2. Beschäftigt euch mit den beiden Transformationen `mutate()` und `summarize()`. Was machen beide und was unterscheiden sie? Erklärt, was der Unterschied im Ergebnis zwischen den beiden folgenden Befehlen ist.

3. Welche Abkürzung gibt es für die obige Anwendung von `summarize()`?

4. Findet heraus, welchen Effekt ein vorangestelltes `group_by()` auf die beiden Transformationen `mutate()` und `summarize()` haben kann. Testet das anhand der Variable für das Geschlecht der Probanden.

5. Welche Visualisierungsform würdet ihr für die Häufigkeiten der Bundesländer, F3, verwenden? Wie könnt ihr die Häufigkeiten schnell visuell darstellen, ohne manuell gruppieren zu müssen?

6. Erstellt eine Gruppierung nach den beiden Merkmalen “Bundesland”, F3, und “Geschlecht”, F4. Ermittelt die Häufigkeiten pro Gruppe.

7. Stellt die Häufigkeiten in den Gruppen aus Aufgabe 6 visuell dar. Welche Form ist dafür geeignet?

8. Stellt nun die *relative* Häufigkeit der Geschlechter für jedes Bundesland visuell dar.

9.2. Schritt 2: Gruppierte Analysen und Visualisierungen

In diesem Abschnitt verbindet ihr gruppierte Auswertungen mit metrischen und ordinalen Variablen. Ziel ist es, nicht nur Häufigkeiten, sondern auch Mittelwerte und Verteilungen in Teilgruppen sinnvoll zu beschreiben.

9. Wie ist die Preisbereitschaft für Milch der Marke Weihenstephan, F13, in den Bundesländern, F3, verteilt?

10. Wie ist die mittlere Preisbereitschaft für Milch der Marke Weihenstephan, F13, in den Bundesländern, F3, pro Geschlecht, F4?

11. Unterscheidet sich die mittlere Preisbereitschaft, F13, zwischen Probanden, die Lebensmittel auf dem Wochenmarkt, F8, einkaufen und denen, die das nicht tun?

12. Wie ist die mittlere Zustimmung der Probanden nach Bundesland, F3, zur folgenden Aussage, F11, “Ich kaufe regionale Lebensmittel, um die regionale Wirtschaft zu unterstützen”?

13. In welchem Bundesland, F3, ist die mittlere Zustimmung zur Aussage, F12, “Ich esse gerne Essen aus anderen Kulturen” am höchsten?

14. Wie ist der Preis für Milch der Marke Weihenstephan, bei dem das Produkt noch als günstig eingeschätzt wird, F16, über die Altersgruppen, F2, verteilt?

15. Wie ist die mittlere Mediennutzung, F216, in den unterschiedlichen Altersklassen, F2, ausgeprägt?

Literaturverzeichnis